# Projet ACOL

Alexandre, Samir, Nathanaël, Louis

## Table des matières

I. Cahier des charges ..... 2
Description ..... 2
Contexte métier ..... 3
Fonctionnalités ..... 3
Menu principal ..... 3
Nouvelle partie ..... 3
Gestion des objets et de l'inventaire ..... 3
Pause ..... 4
Sauvegarde et chargement ..... 4
Configurations et paramètres ..... 4
Musiques et ambiance sonore ..... 5
Niveaux de difficulté ..... 5
Aide et règles ..... 5
Fin de partie ..... 5
Interactions avec d'autres personnages ..... 5
Déplacement du personnage ..... 6
II. Cas d'utilisation ..... 6
Cas d'utilisation général ..... 6
Exploration d'un bâtiment ..... 7
Pré-conditions ..... 7
Cas d'utilisation ..... 7
Rencontre avec un personnage hostile ..... 9
Pré-conditions ..... 9
Cas d'utilisation ..... 9
Rencontre avec un personnage passif ..... 11
Pré-conditions ..... 11
Cas d'utilisation ..... 11
Pause du jeu ..... 14
Pré-conditions ..... 14
Cas d'utilisation ..... 14
Navigation dans le menu principal ..... 18
Pré-conditions ..... 18
Cas d'utilisation ..... 18
Interagir avec un coffre ..... 21
Pré-conditions ..... 21
Cas d'utilisation ..... 21
Le joueur ouvre l'inventaire ..... 23
Pré-conditions ..... 23
Cas d'utilisation ..... 23
Fin de partie ..... 28
Pré-conditions ..... 28
Cas d'utilisation ..... 28
Ramasser un objet ..... 30

Pré-conditions ..... 30
Cas d'utilisation ..... 30
III. Diagramme de classes d'analyse ..... 32
IV. Architecture logique du logiciel ..... 33
Package Model ..... 33
Package Controller ..... 33
Package View ..... 33
Package Utils (Observer) ..... 33
Comment ajouter une interface graphique ..... 34
V. Conception de l'incrément ..... 34
Incrément choisi ..... 34
Diagramme de classes logicielles ..... 35
Vue d'ensemble MVC ..... 35
Package model.core ..... 36
Package model.command ..... 37
Package model.world ..... 38
Package model.inventory ..... 39
Package model.entity ..... 40
Diagramme d'états-transitions ..... 41
Diagrammes de séquence ..... 42
Initialisation d'une partie ..... 42
Consommation de nourriture ..... 43
Ramasser un objet ..... 44
Déverouiller un bâtiment ..... 45
Déplacement entre zones.png ..... 46
VI. Manuel utilisateur ..... 47
Lancement ..... 47
Navigation générale ..... 47
Menu principal ..... 48
Écran de jeu ..... 48
Inventaire ..... 48
Pause ..... 48
VII. Bilan sur les outils de modélisation ..... 48
PlantUML ..... 48

# I. Cahier des charges

## Description

Le jeu met en scène un étudiant de l'ENSIMAG à la recherche de son diplôme.

Le but du jeu est de trouver ce diplôme en explorant un campus parsemé d'embûches. Le bâtiment contenant le diplôme est verrouillé, le joueur doit trouver la bonne clef afin d'y accéder et de gagner le jeu. Le joueur doit explorer 4 bâtiments obligatoires avant de pouvoir accéder au diplôme, chaque bâtiment contenant la clef pour le bâtiment suivant (le 4ème contient la clef pour le bâtiment final). Chacun de ces bâtiments est protégé par un ennemi puissant. Le joueur possède aussi la possibilité d'explorer d'autres bâtiments optionnels pouvant contenir des coffres, des ennemis et des personnages passifs. Les bâtiments sont reliés entre eux par des rues.

Sur le chemin, le joueur pourra rencontrer des ennemis, trouver des objets (armes, nourriture, clefs, armures, power-ups), rencontrer des alliés, effectuer des quêtes et marchander.

La partie se termine lorsque le joueur trouve le trésor (victoire) ou lorsque sa vie tombe à 0 (défaite).

## Contexte métier

Le jeu s'adresse à un public de joueurs occasionnels.

Le jeu est conçu pour être utilisé sur un ordinateur personnel.

Les ressources matérielles nécessaires sont :
- Un ordinateur personnel (portable ou fixe)
- Un clavier afin de contrôler le jeu
- Un écran
- Éventuellement un système audio pour profiter de la musique et des effets sonores

## Fonctionnalités

Le système couvre les fonctionnalités suivantes :
- menu principal
- création d'une nouvelle partie
- chargement d'une sauvegarde
- gestion d'un inventaire
- pause
- sauvegarde et chargement
- paramètres de jeu
- gestion des musiques et effets sonores
- niveaux de difficulté
- écran de fin de partie.

Ces fonctionnalités sont détaillées ci-dessous.

### Menu principal

Au démarrage, l'application doit afficher un menu principal permettant au minimum :
- de commencer une nouvelle partie
- de charger une partie sauvegardée
- d'accéder aux paramètres
- de consulter l'aide ou les règles
- de quitter l'application.

### Nouvelle partie

L'utilisateur doit pouvoir créer une nouvelle partie en choisissant plusieurs paramètres :
- nom du joueur ou profil
- niveau de difficulté

### Gestion des objets et de l'inventaire

L'utilisateur peut ramasser des objets lorsqu'il explore le campus. Si l'utilisateur se déplace sur un objet il peut alors le ramasser. Il possède un inventaire avec une taille limitée, s'il est plein il ne peut plus ramasser d'objet. Il a la possibilité de jeter un objet de son inventaire pour faire de la place.

L'inventaire doit pouvoir permettre de :
- équiper ou déséquiper un objet, il possède un emplacement pour une arme, un emplacement pour une armure et un emplacement pour un power-up
- consommer de la nourriture pour se soigner
- consulter la description d'un objet
- jeter un objet

- consulter leur rôle ou leur usage

L'utilisateur a accès à différents types d'objets différents :
- des armes faisant plus ou moins de dégâts
- de l'armure protégeant le joueur
- des clefs permettant d'ouvrir différentes portes ou portes, une fois utilisée sur une porte la clef disparaît
- de la nourriture permettant de se soigner lorsque consommée
- des powers-up améliorant certaines caractéristiques
- des objets utilitaires ou de quête

Les objets sont répartis aléatoirement sur le campus, certains peuvent être trouvés dans des coffres, d'autres à terre. Certains objets sont plus rare que d'autres.

**Pause**

Le système doit proposer une fonctionnalité de pause accessible pendant la partie. Lorsque le jeu est en pause, l'utilisateur doit pouvoir :
- reprendre la partie
- sauvegarder la partie
- consulter l'aide
- modifier les paramètres
- revenir au menu principal après confirmation
- quitter l'application après confirmation.

Pendant la pause, l'état du jeu doit être figé. Aucun déplacement, événement ou interaction ne doit progresser tant que la partie n'est pas reprise.

**Sauvegarde et chargement**

Le système doit permettre la sauvegarde manuelle d'une partie et le chargement d'une sauvegarde existante.

Une sauvegarde doit contenir :
- la position du personnage
- l'état de la carte
- les bâtiments déjà explorés
- l'inventaire
- les ressources du joueur
- l'état des personnages hostiles ou événements persistants
- le niveau de difficulté
- la date ou l'heure de sauvegarde.

Le chargement doit permettre de restaurer fidèlement l'état précédent du jeu.

**Configurations et paramètres**

Le système doit être configurable. Les paramètres doivent être centralisés afin de faciliter leur compréhension et leur modification.

Les paramètres doivent inclure au minimum :
- volume de la musique
- volume des effets sonores
- activation ou désactivation de la musique
- activation ou désactivation des bruitages
- vitesse d'animation ou de défilement si nécessaire

- mode d'affichage
- confirmation avant quitter ou écraser une sauvegarde

**Musiques et ambiance sonore**

Le jeu doit gérer une ambiance sonore minimale comprenant :
- une musique de menu
- une musique ou ambiance de jeu
- des effets sonores associés aux événements importants
- la possibilité de couper indépendamment musique et effets sonores.

Les préférences audio doivent être mémorisées d'une session à l'autre.

**Niveaux de difficulté**

Le jeu doit proposer trois niveaux de difficulté :
- Facile
- Moyen
- Difficile.

La difficulté doit influencer de manière concrète l'expérience de jeu, par exemple :
- agressivité des personnages hostiles
- quantité ou rareté des ressources
- niveau d'aide fourni au joueur
- fréquence des dangers
- résistance ou puissance des ennemis.

**Aide et règles**

L'application doit proposer une section d'aide accessible depuis le menu principal et depuis la pause.

Cette aide doit présenter :
- l'objectif du jeu
- les commandes
- le fonctionnement de l'inventaire
- le rôle des objets principaux
- les conditions de victoire et d'échec
- les effets des niveaux de difficulté.

**Fin de partie**

Le jeu doit prévoir un écran de fin de partie affichant au minimum :
- le résultat de la partie
- la victoire ou la défaite
- la cause de fin de partie
- le temps de jeu

Depuis cet écran, l'utilisateur doit pouvoir :
- recommencer une partie
- revenir au menu principal
- quitter l'application.

**Interactions avec d'autres personnages**

Le joueur peut rencontrer différents personnages dans les rues ou dans les bâtiments, ils sont plus ou moins hostiles.

Certains personnages sont passifs. Ils permettent d'acquérir des objets en échange de quêtes ou d'autres objets.

Si le PNJ possède une quête, le joueur a la possibilité de l'accepter. Il recevra la récompense après avoir atteint l'objectif de la quête et avoir interagi à nouveau avec le PNJ.

Une quête peut avoir comme objectif :
- récupérer un objet
- tuer un ennemi
- explorer un bâtiment

Certains personnages sont des créatures hostiles. Ils attaquent le joueur si celui-ci est dans le champ de vision de la créature. Un duel se déroule de la sorte :
- Le joueur qui est trop proche rentre en duel avec la créature.
- Le combat se déroule au tour par tour.
- Le joueur et la créature ont une barre de vie et une barre d'endurance.
- Chaque attaque coûte de l'endurance à l'attaquant et de la vie au défenseur.
- Au début de chaque tour, le joueur et la créature récupèrent une petite partie d'endurance.

Le premier des deux dont la vie tombe à zéro perd le duel et une partie, ou la totalité des objets qu'il possédait.

**Déplacement du personnage**

L'utilisateur peut faire déplacer le personnage en utilisant les touches Z pour aller à la case au dessus de lui, Q pour aller à la case de gauche, D pour aller à la case de droite et S pour aller à la case en dessous de lui.

Si l'utilisateur essaye de faire déplacer le personnage sur une zone étant un bâtiment ou sur une case contenant un coffre, ou un PNJ, alors son déplacement ne se fera pas.

Si l'utilisateur se déplace sur un objet il peut alors le ramasser. L'utilisateur peut aussi faire déplacer le personnage dans les zones étant des bâtiments et pour se faire il doit déverrouiller la zone du bâtiment sinon il ne peut pas accéder à la zone.

# II. Cas d'utilisation

Concernant les diagrammes de séquence :

Lorsque les flèches sont encadrées par une boite de la forme

```
alt <cond1>

----
<cond2>
```

les 1ères interactions ne sont effectuées que lorsque `<cond1>` est vraie, les 2èmes que si `<cond2>` est vraie.

Lorsqu'elles sont encadrées par une boite de la forme `opt <cond>` les interactions ne sont effectuées que lorsque `<cond>` est vraie.

## Cas d'utilisation général

Pré-conditions :
- Avoir lancé l'application

1. Le joueur est dans le menu d'accueil
   A. **Le joueur est dans le menu**
   B. Le joueur lance une partie
      a. Le joueur se balade dans le campus et tombe face à un personnage hostile
         i. **Rencontre avec un personnage hostile, un combat se lance**
            i. Le joueur perds le combat et tombe à 0 point de vie, il meurt et perds la partie.
      b. Le joueur se balade sur la carte et tombe face à un bâtiment
         i. **Exploration d'un bâtiment**
      c. Le joueur se balade sur la carte et tombe face à un personnage hostile
         i. **Rencontre avec un personnage hostile**
            i. Survie
            ii. Mort/Défaite
      d. Le joueur tombe sur le trésor, possède la bonne clé ( pour avoir la bonne clé il a visité les 4 bâtiments ), et interagit avec le coffre
         i. Victoire, le joueur a donc gagné et la partie est terminé.

## Exploration d'un bâtiment

**Pré-conditions**

- Le joueur est dans une partie
- Le joueur est à portée d'un bâtiment

**Cas d'utilisation**

1. Le joueur essaye de rentrer dans un bâtiment en appuyant sur la touche interagir
   A. La porte du bâtiment est verrouillé
      a. Le joueur possède une clef
         i. La clef est consommée et le bâtiment est déverrouillé
      b. Le joueur n'a pas de clef valide pour ce bâtiment
         i. la porte du bâtiment reste verrouillée, le joueur ne peut pas rentrer, un message apparaît indiquant que le batiment est verrouillé
   B. La porte du bâtiment n'est pas verrouillé
      a. Le joueur rentre dans le bâtiment

**Diagramme(s) de séquence**

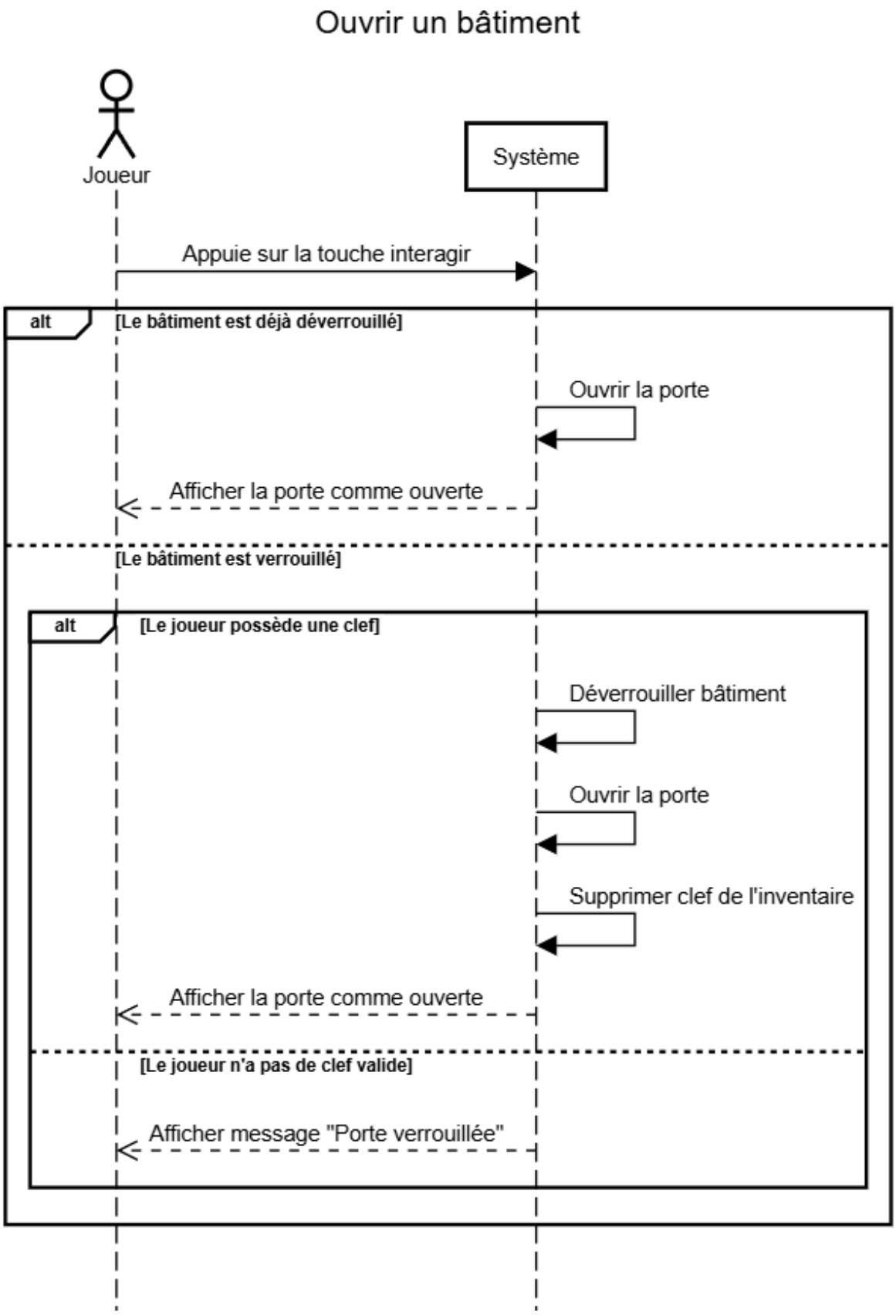



Fig. 1. – Ouvrir un bâtiment

## Rencontre avec un personnage hostile

### Pré-conditions

- Le joueur est dans une partie
- Le joueur est à portée du personnage

### Cas d'utilisation

1. Le duel commence. Tour après tour, le joueur et l'ennemi s'affrontent. Leurs inventaires sont visibles. Ils choisissent tour par tour une arme avec laquelle attaquer. Le niveau de difficulté détermine l'arsenal de l'ennemi. Le personnage dont c'est le tour perd des points de l'endurance, et l'autre joueur perd de la vie. Chaque arme inflige un nombre aléatoire de dégâts et consomme un nombre aléatoire d'endurance, tous deux compris dans des fourchettes liées aux statistiques de l'arme utilisée.
   A. La vie du joueur tombe à zéro en premier
      a. Le joueur meurt
   B. La vie de l'ennemi tombe à zéro en premier
      a. L'ennemi meurt, le personnage hostile disparaît et ses objets apparaissent à la case ou était le personnage hostile, le combat est terminé

**Diagramme(s) de séquence**

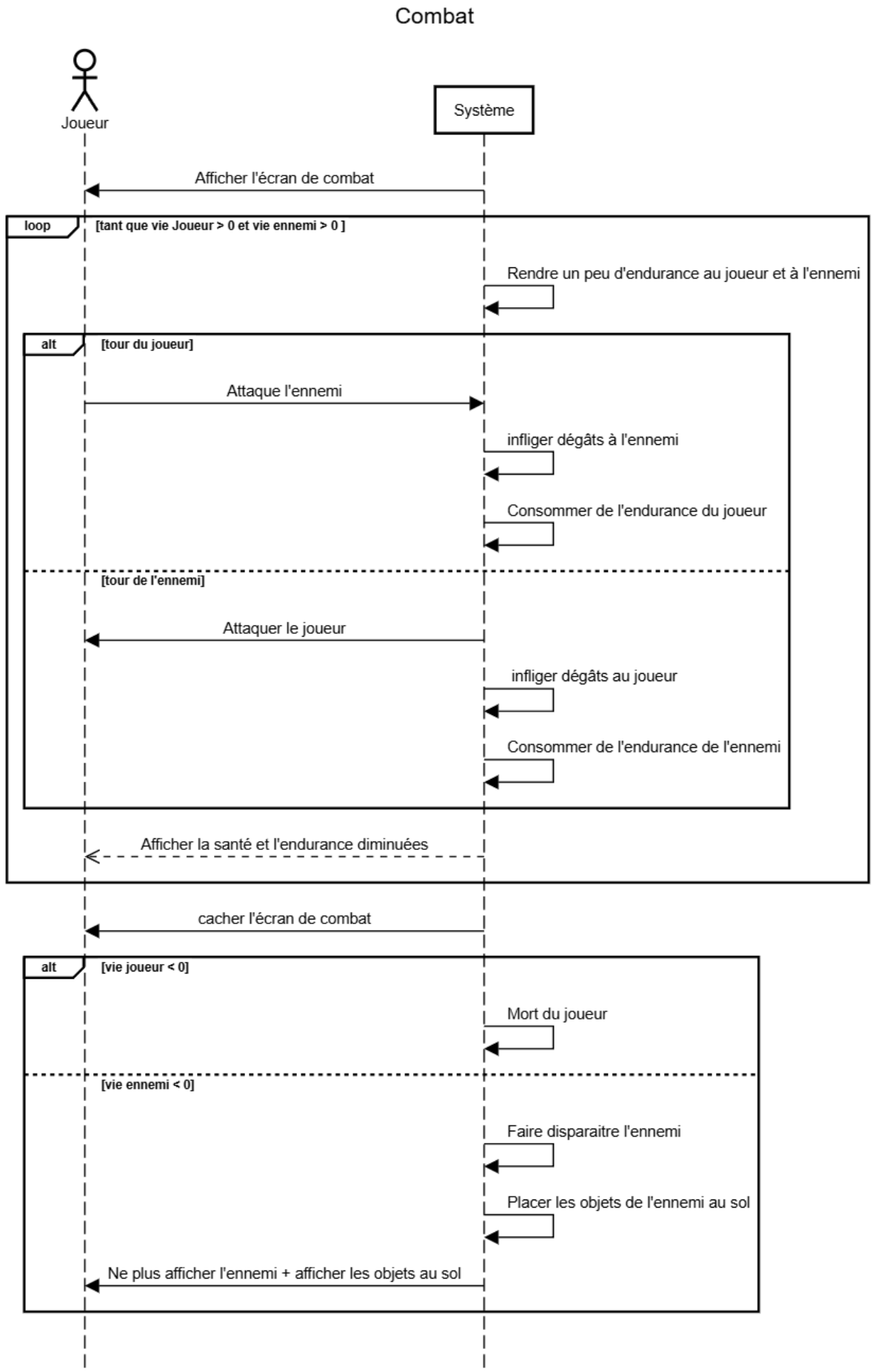

Combat
Joueur
Système
Afficher l'écran de combat
loop
[tant que vie Joueur > 0 et vie ennemi > 0 ]
Rendre un peu d'endurance au joueur et à l'ennemi
alt
[tour du joueur]
Attaque l'ennemi
infliger dégâts à l'ennemi
Consommer de l'endurance du joueur
[tour de l'ennemi]
Attaquer le joueur
infliger dégâts au joueur
Consommer de l'endurance de l'ennemi
Afficher la santé et l'endurance diminuées
cacher l'écran de combat
alt
[vie joueur < 0]
Mort du joueur
[vie ennemi < 0]
Faire disparaitre l'ennemi
Placer les objets de l'ennemi au sol
Ne plus afficher l'ennemi + afficher les objets au sol

Fig. 2. – Combat

## Rencontre avec un personnage passif

### Pré-conditions

- Le joueur est dans une partie
- Le joueur est à portée du personnage (PNJ)

### Cas d'utilisation

1. Le joueur interagit avec le personnage passif
2. Le PNJ est un commerçant
   A. Le joueur a la place dans son inventaire et les objets demandés pour réaliser l'échange
      a. L'échange est fait. Les ressources demandées sont retirées de l'inventaire du joueur et celles proposés y sont ajoutées.
   B. Le joueur n'a pas de place ou n'a pas les objets nécessaires
      a. L'échange n'est pas réalisé et un message est affiché
3. Le PNJ possède une quête
   A. Le joueur accepte la quête
      a. La quête est ajoutée à la liste des quêtes du joueur et le PNJ ne possède désormais que des lignes de dialogue
   B. Le joueur refuse la quête
      a. Le PNJ reste tel quel
4. Le PNJ ne possède que des lignes de dialogues
   A. Le joueur peut continuer d'interagir pour poursuivre le dialogue ou quitter

### Diagramme(s) de séquence

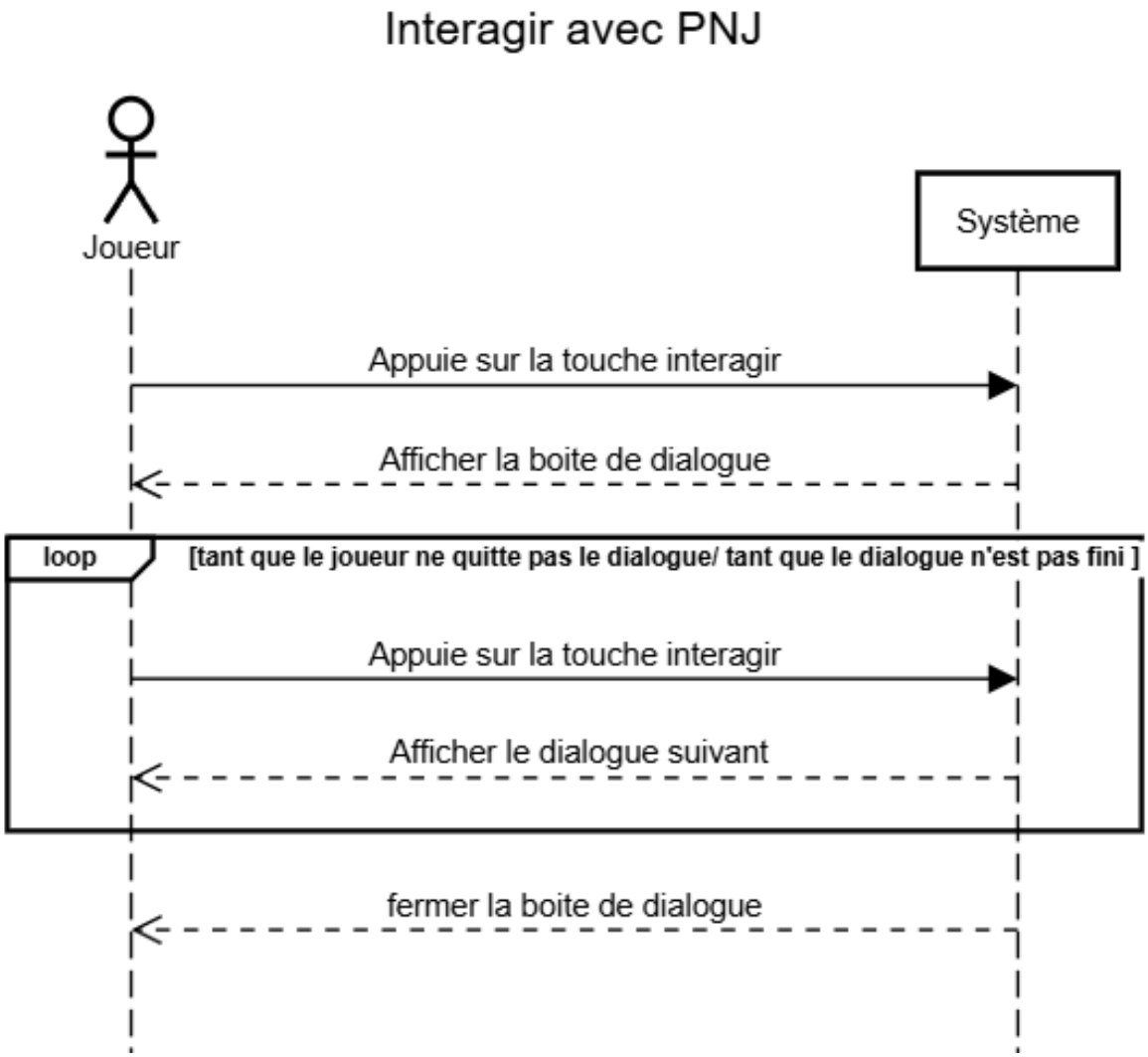



Fig. 3. – Interagir avec un PNJ

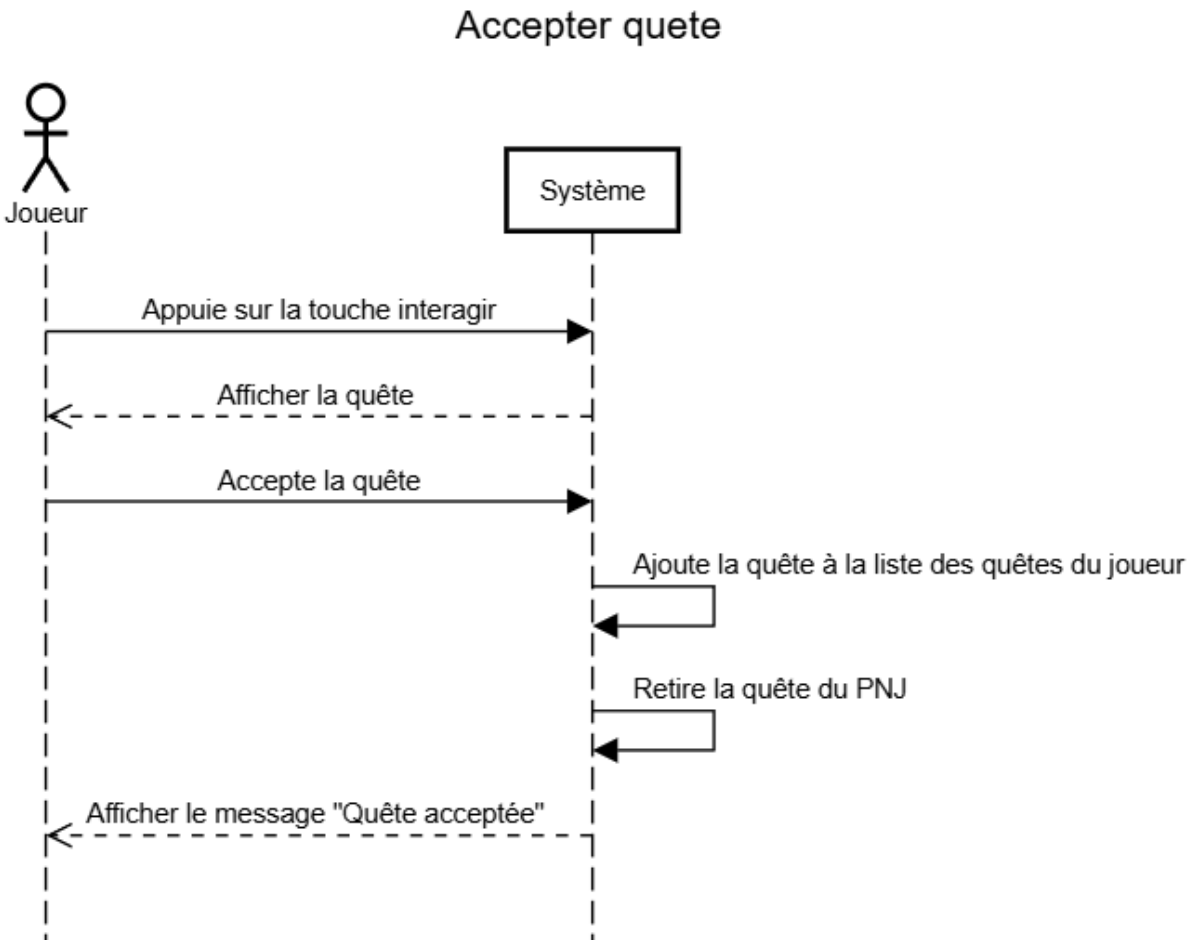



Fig. 4. – Accepter une quête

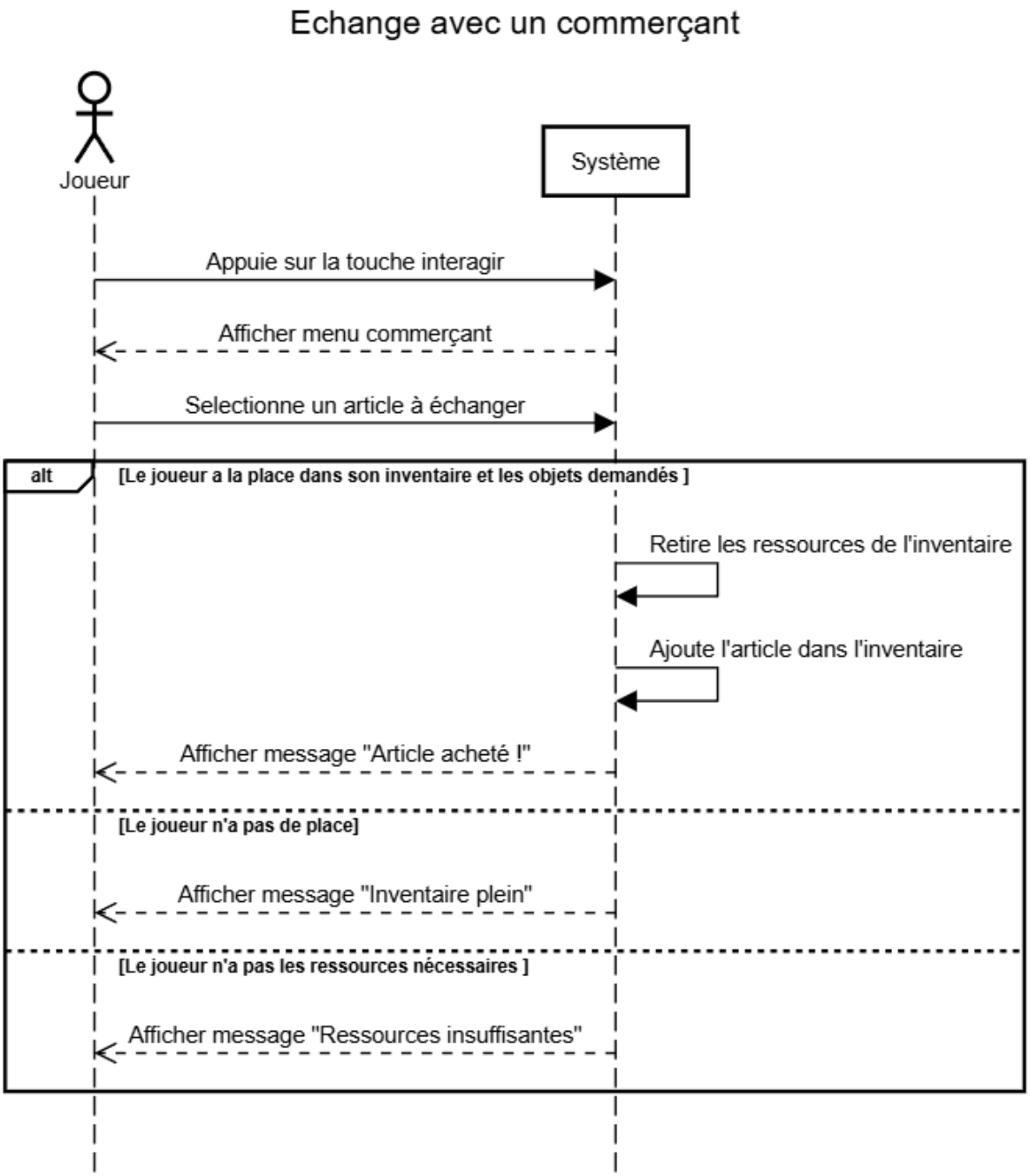



Fig. 5. – Échange avec un commerçant

## Pause du jeu

### Pré-conditions

- Le joueur est dans une partie

### Cas d'utilisation

1. Appuie sur la touche pause
2. Un menu s'affiche
   A. L'utilisateur peut reprendre la partie
   B. Sauvegarder la partie
   C. Consulter l'aide
   D. Modifier des paramètres de configurations

E. Revenir au menu principal
    a. Confirmer
    b. Annuler
F. Quitter le jeu
    a. Confirmer
    b. Annuler
    c. Sauvegarder et quitter

**Diagramme(s) de séquence**

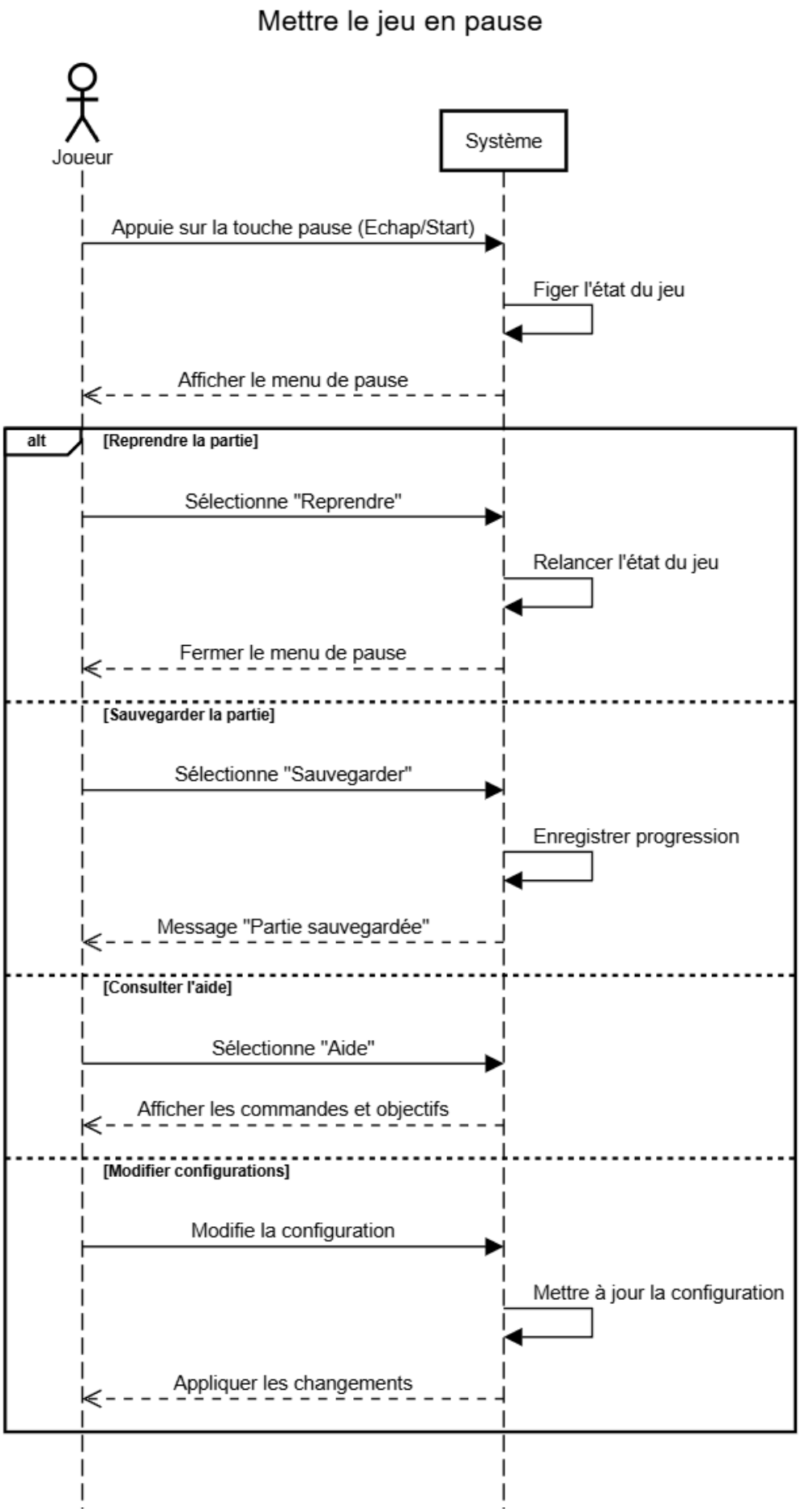



Fig. 6. – Mettre le jeu en pause

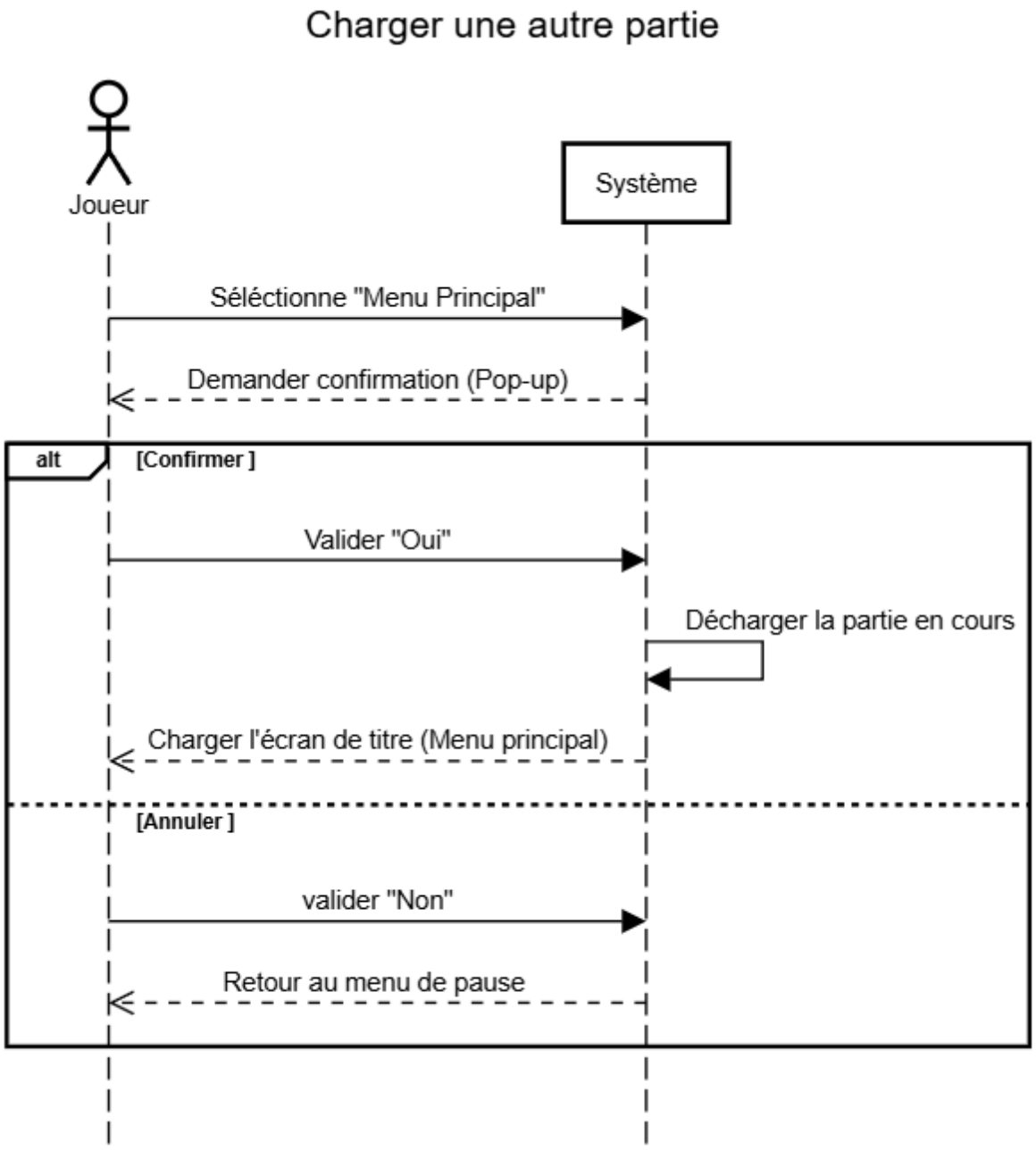



Fig. 7. – Charger une autre partie

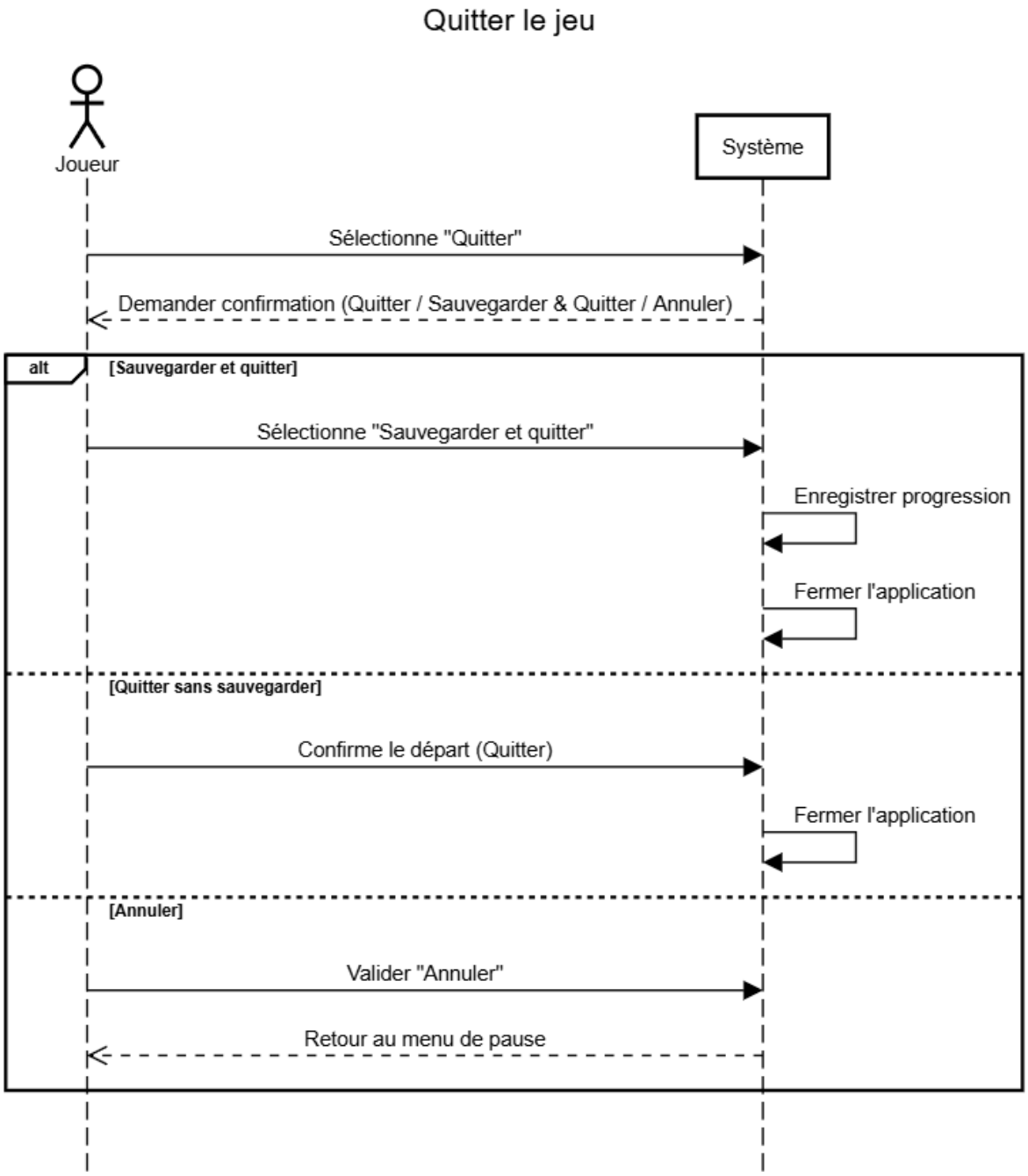



Fig. 8. – Quitter le jeu

## Navigation dans le menu principal

### Pré-conditions

- Le joueur est dans le menu principal

### Cas d'utilisation

1. Commencer une nouvelle partie
2. Charger une ancienne sauvegarde
3. Accéder aux paramètres du jeu
4. Consulter l'aide
5. Quitter le jeu
   A. Confirmer

B. Annuler

**Diagramme(s) de séquence**

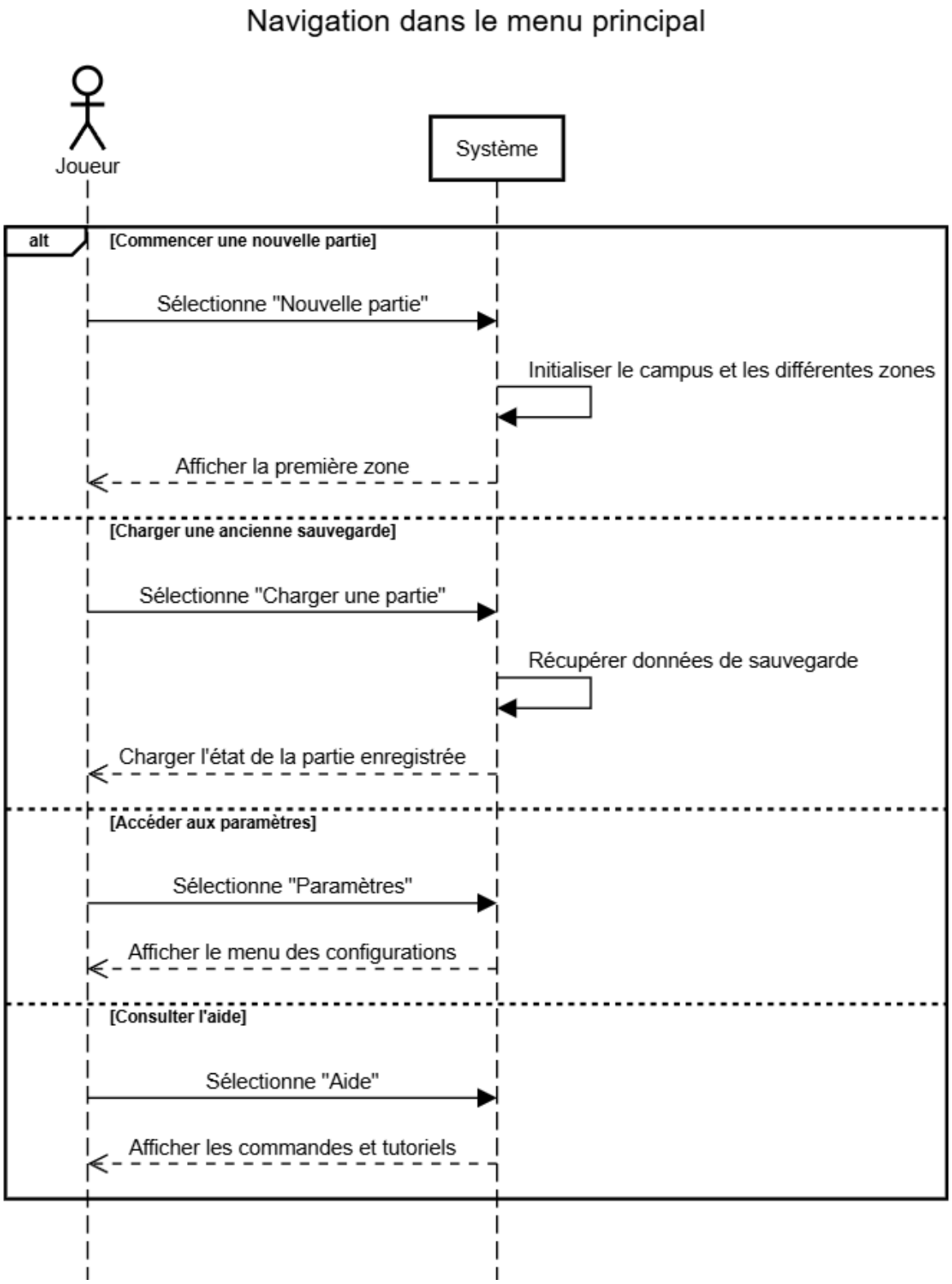



Fig. 9. – Navigation dans le menu principal

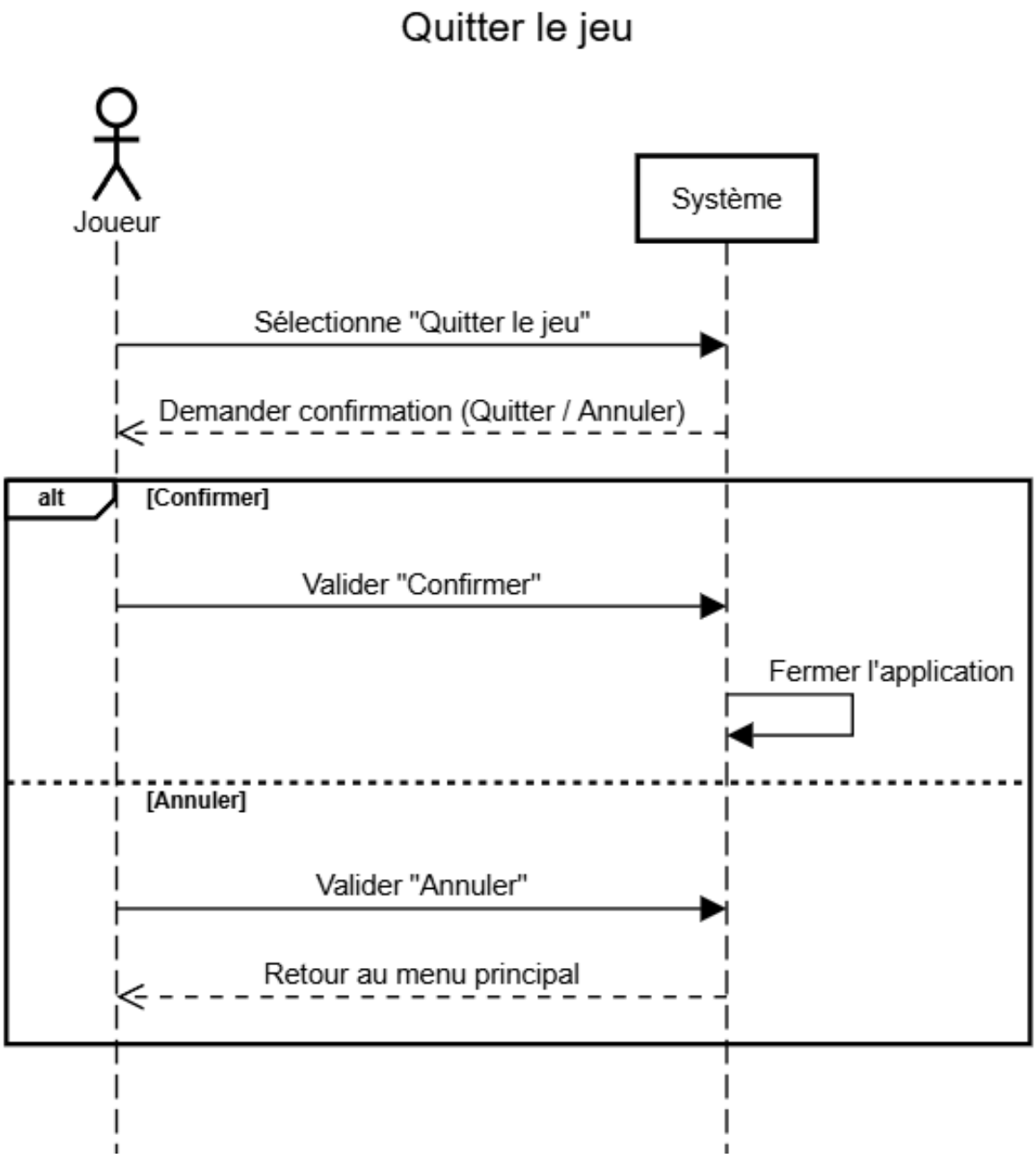



Fig. 10. – Quitter le jeu

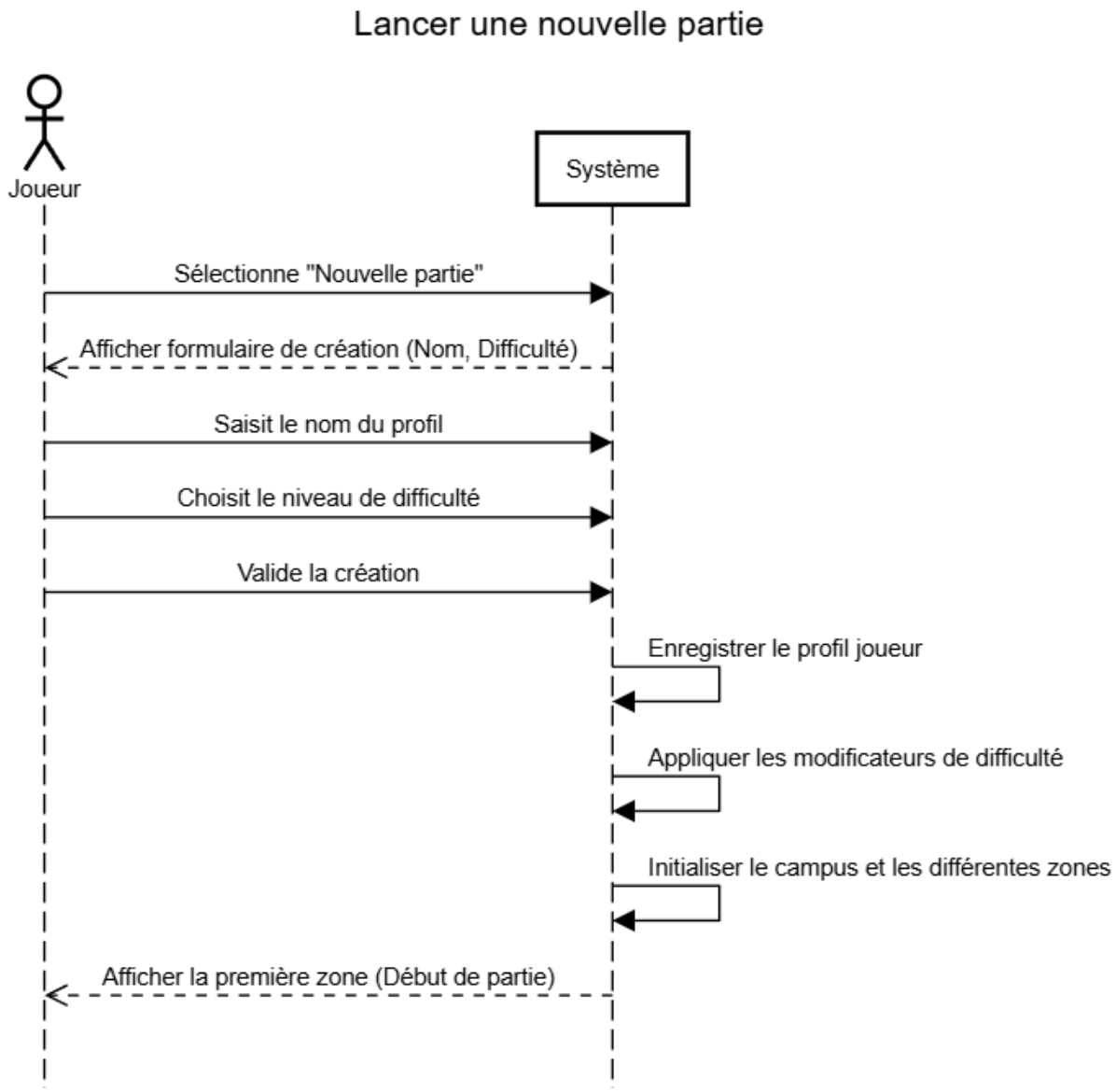



Fig. 11. – Lancer une nouvelle partie

## Interagir avec un coffre

### Pré-conditions

- Le joueur est dans une partie
- Le joueur est à portée du coffre

### Cas d'utilisation

1. Le joueur essaye d'ouvrir le coffre en appuyant sur la touche interagir
   A. Le coffre est verrouillé
      a. Le joueur possède une clef capable d'ouvrir le coffre
         i. La clef est consommée et le coffre est déverrouillé, le coffre disparaît et l'objet contenu dans le coffre apparaît
      b. Le joueur n'a pas de clef capable de déverrouiller pour le coffre
         i. le coffre reste verrouillée, le joueur ne peut pas l'ouvrir, un message apparaît indiquant que le coffre est verrouillé
   B. Le coffre n'est pas verrouillé

a. Le joueur interagit avec le coffre, le coffre disparaît et l'objet contenu dans le coffre apparaît à la place du coffre

**Diagramme(s) de séquence**

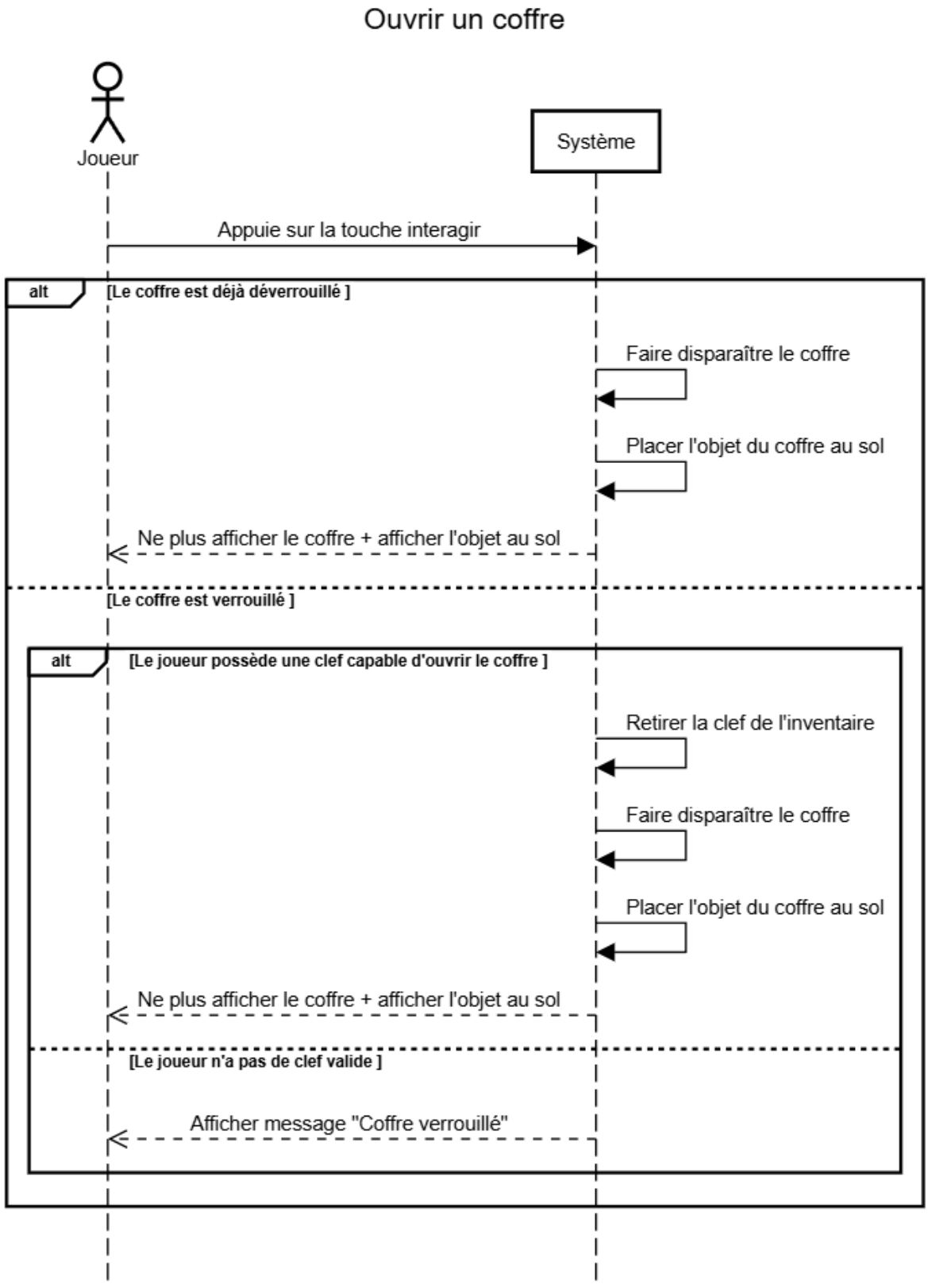



Fig. 12. – Ouvrir un coffre

## Le joueur ouvre l'inventaire

### Pré-conditions

- Le joueur est dans une partie

### Cas d'utilisation

1. Le joueur appuie sur le bouton pour ouvrir l'inventaire, l'inventaire s'ouvre
   A. Le joueur sélectionne un objet dans l'inventaire, un menu s'affiche avec des options
      a. L'objet est une arme/ une armure/ un powerup non équipé
         i. Le joueur peut choisir d'équiper l'arme/ l'armure/ le powerup
            i. Si un objet est déjà équipée celui ci est échangée de place avec l'objet sélectionné
            ii. Si aucune objet n'est équipée l'objet sélectionné va dans le slot correspondant
            iii. Les stats sont modifiées en fonction de ce qui est équipé
      b. L'objet est une arme/ une armure/ un powerup équipé
         i. Le joueur peut choisir de déséquiper l'objet
            i. S'il y a de de la place dans l'inventaire, l'objet est déplacé du slot correspondant à l'inventaire général
               i. Les stats sont modifiées en fonction de ce qui est déséquipé
            ii. S'il n'y pas de place dans l'inventaire, il n'est pas possible de le déséquiper
      c. L'objet est de la nourriture
         i. Le joueur peut choisir de consommer l'objet
            i. Le joueur est soigné, l'objet disparait de l'inventaire
      d. Avec n'importe quel objet, le joueur peut choisir de le jeter
         i. S'il y a de la place devant le joueur, l'objet disparait de l'inventaire et est placé sur la case devant le joueur
         ii. S'il n'y a pas de place devant le joueur, il est impossible de jeter l'objet
      e. Avec n'importe quel objet, le joueur peut choisir de consulter sa description
         i. Un menu s'ouvre affichant une description + stats de l'objet
   B. Le joueur peut fermer l'inventaire

### Diagramme(s) de séquence

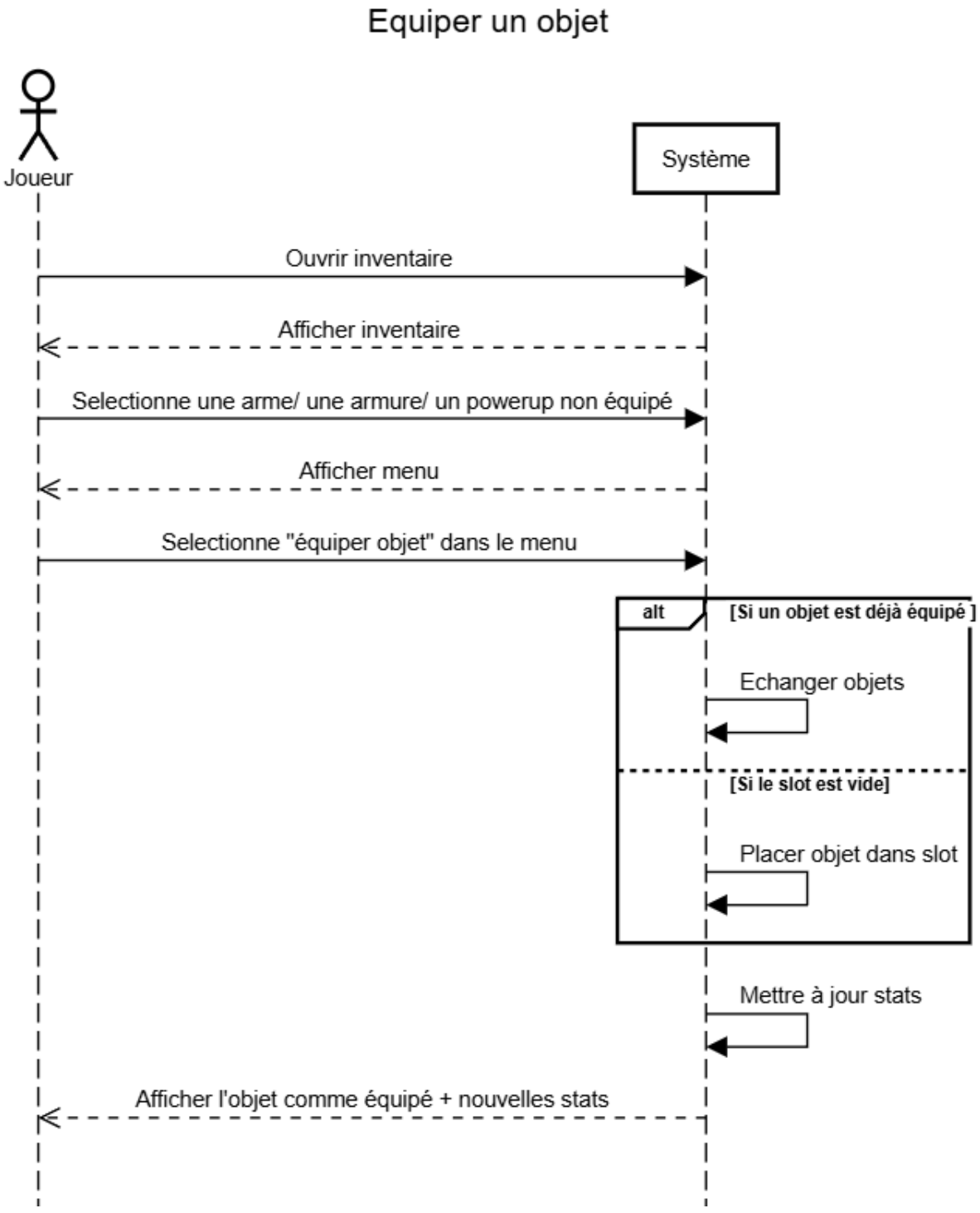



Fig. 13. – Equiper un objet

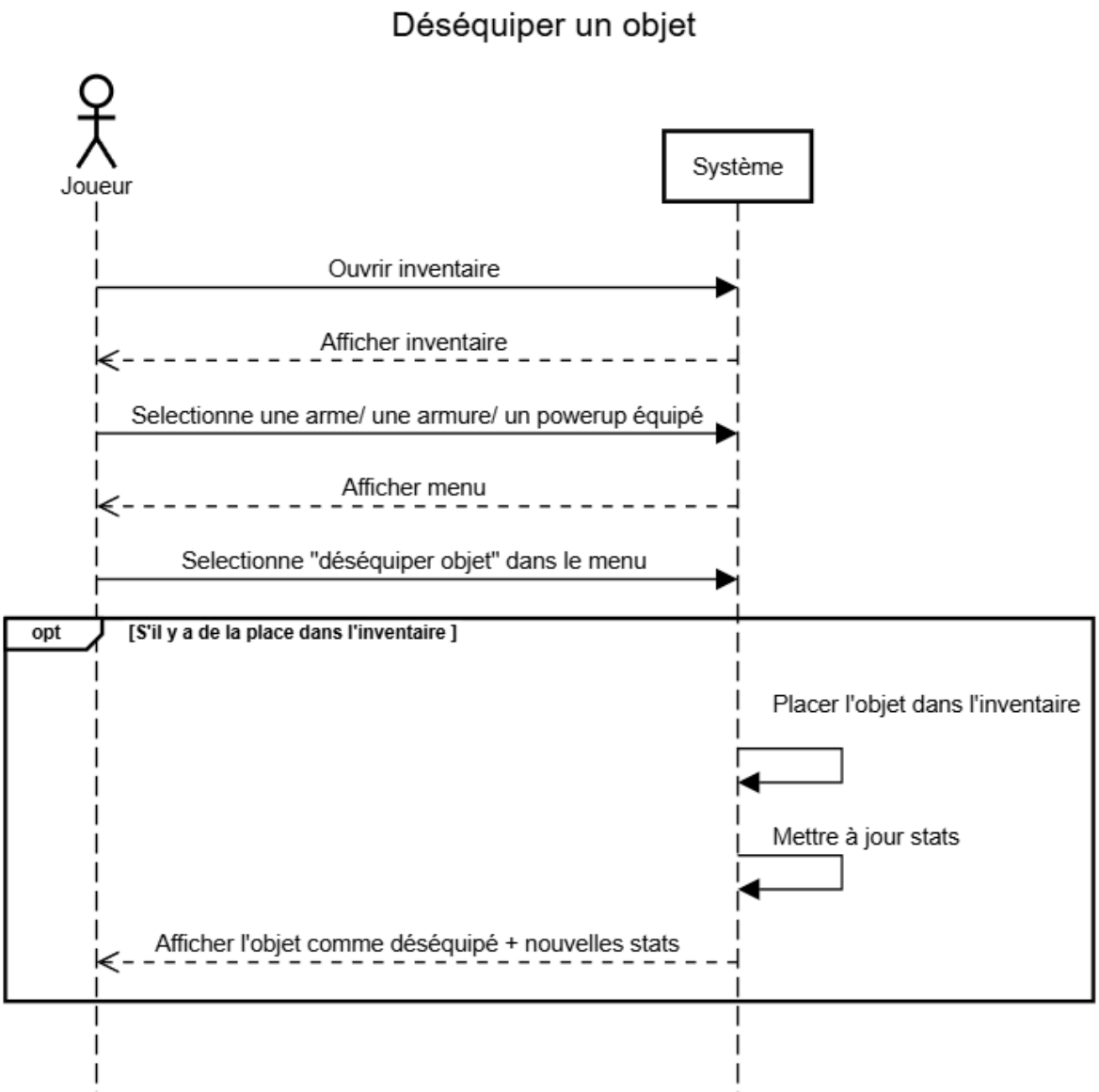



Fig. 14. – Desequiper un objet

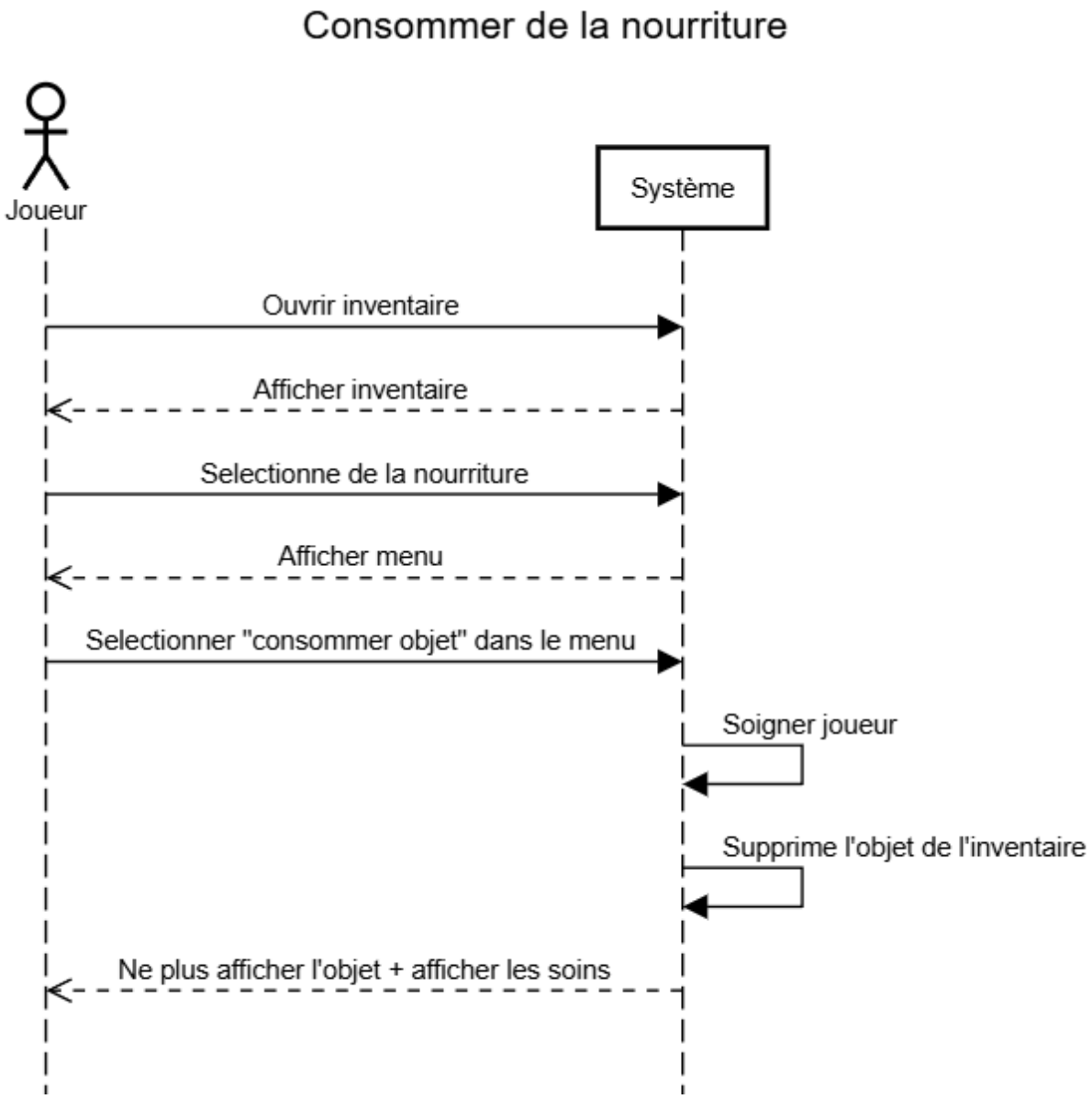



Fig. 15. – Consommer de la nourriture

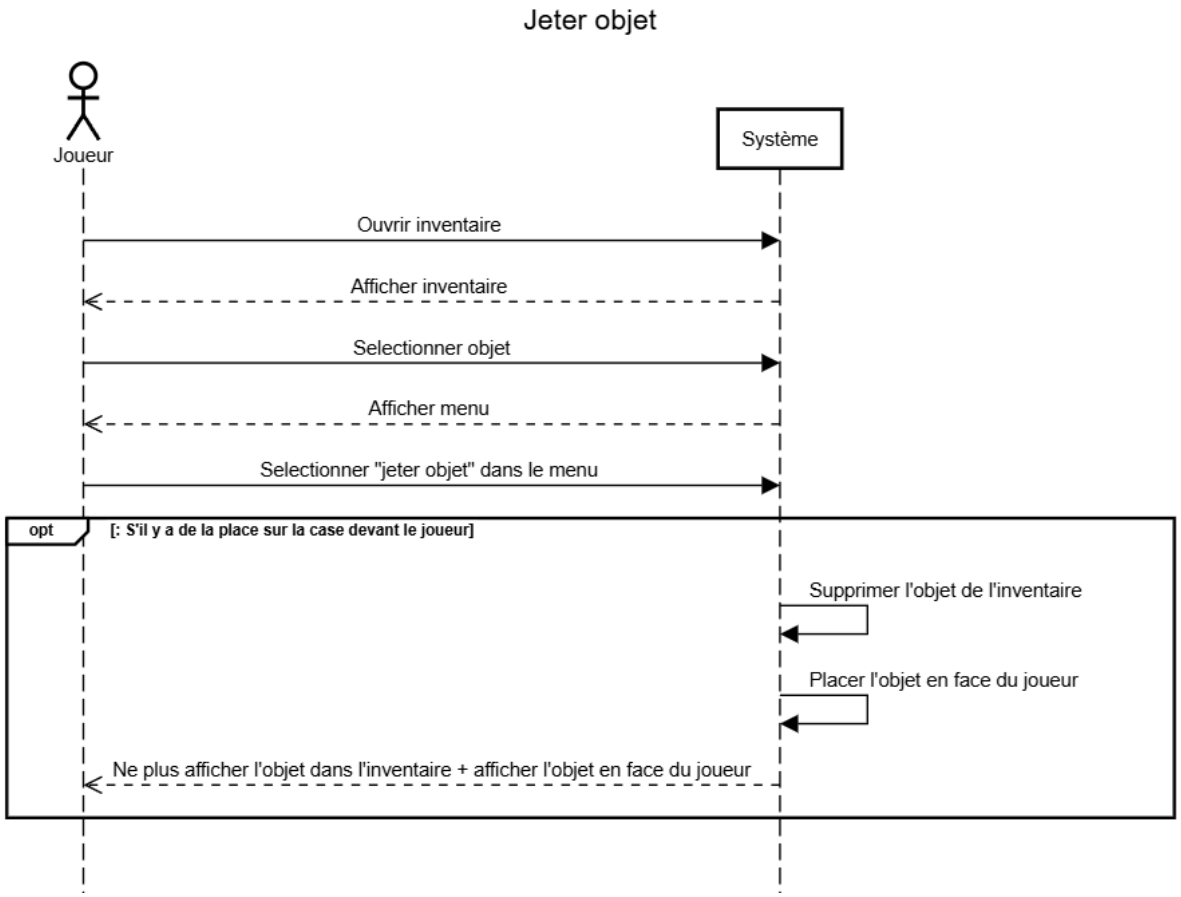



Fig. 16. – Jeter un objet

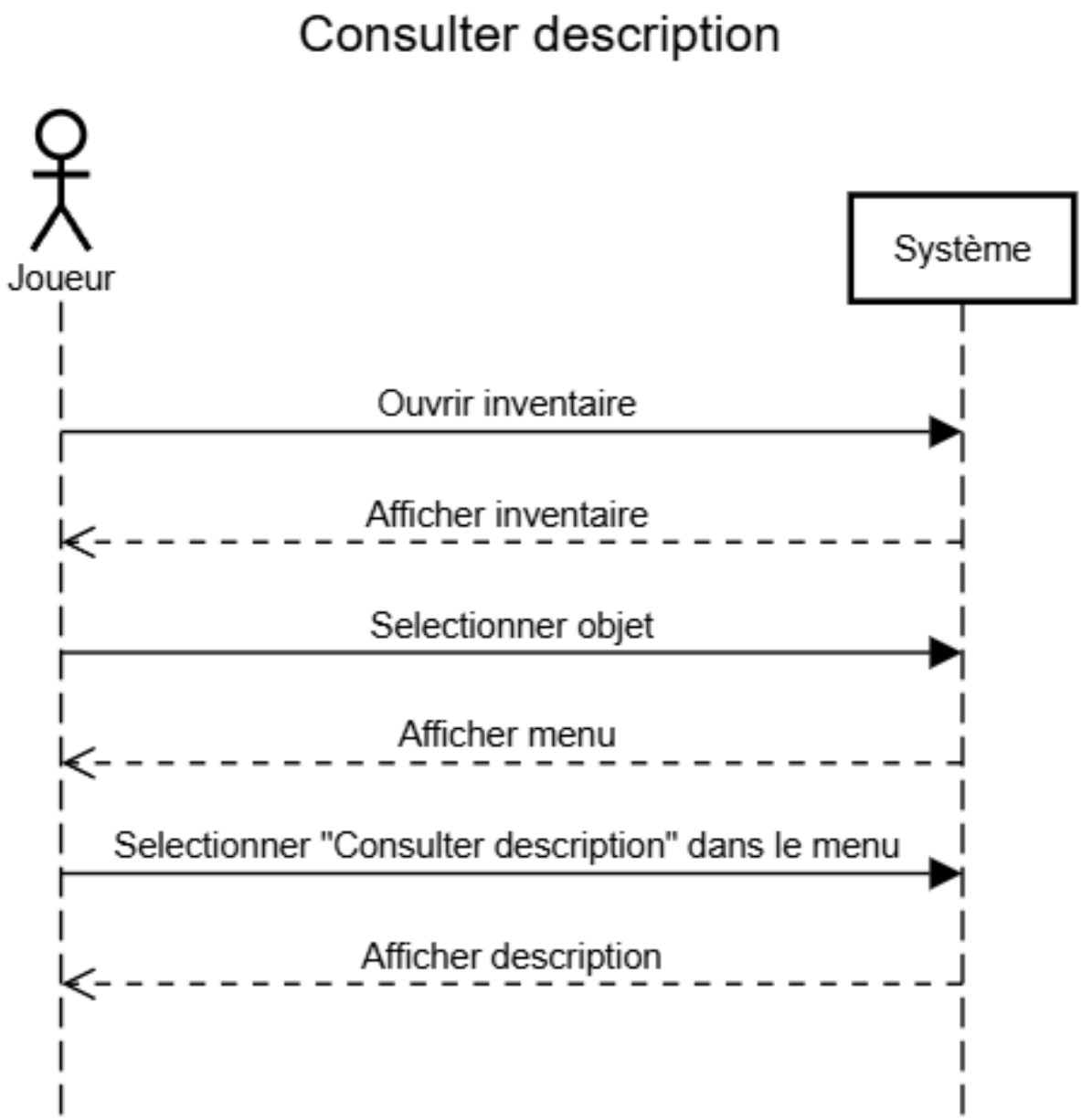



Fig. 17. – Consulter une description

## Fin de partie

**Pré-conditions**

- Le joueur est dans une partie

**Cas d'utilisation**

1. Affichage de l'écran de fin de partie avec: l'issue de la partie, le temps de jeu…
   A. Recommencer une partie
   B. Revenir au menu principal
   C. Quitter le jeu

**Diagramme(s) de séquence**

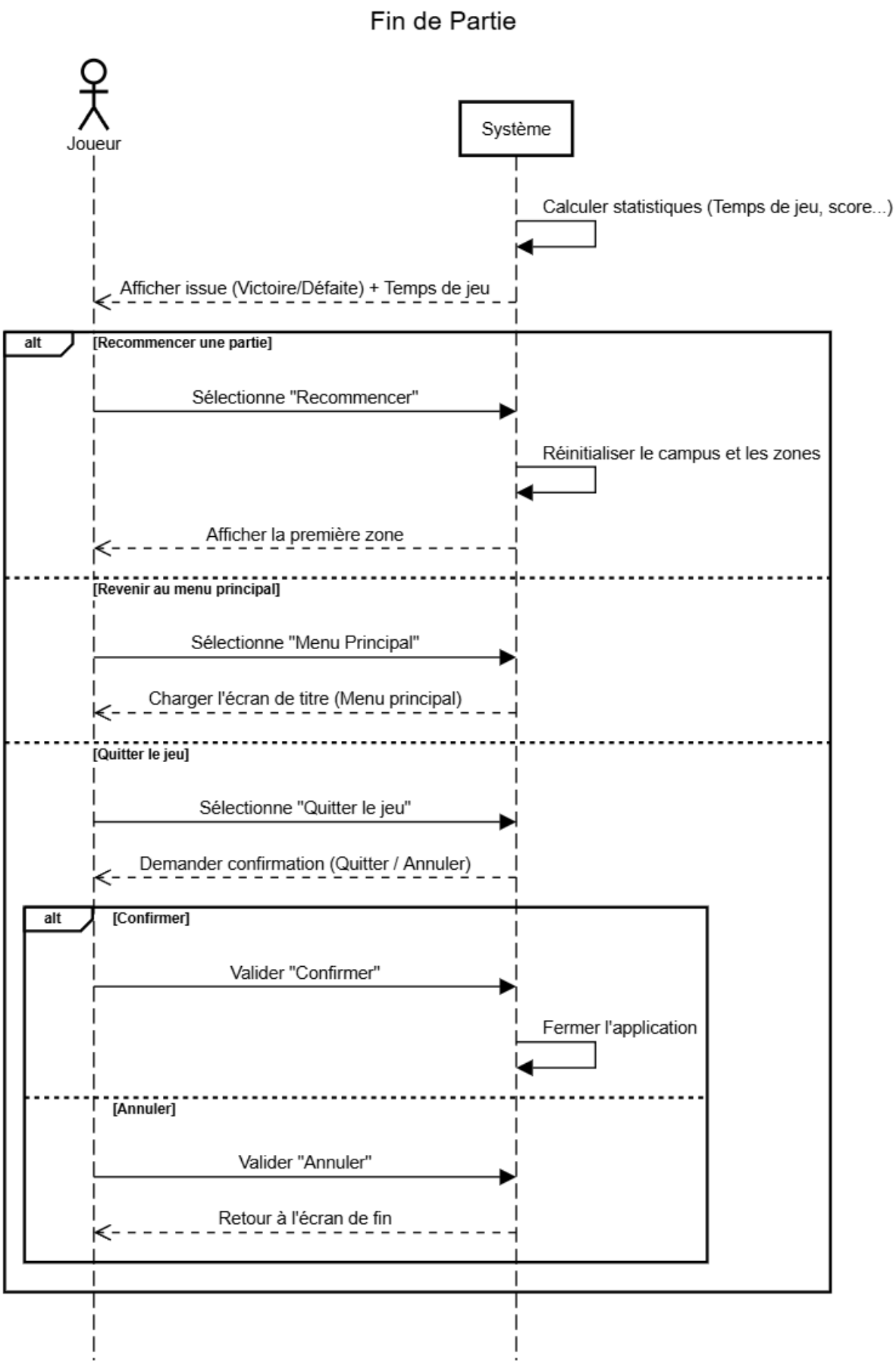



Fig. 18. – Fin de partie

## Ramasser un objet

### Pré-conditions

- Le joueur est dans une partie
- Le joueur est à portée d'un objet

### Cas d'utilisation

1. Le joueur est positionné sur un objet
    A. L'inventaire du joueur n'est pas plein
        a. Le joueur interagit avec l'objet
            i. L'objet va dans l'inventaire du joueur
        b. Le joueur n'interagit pas avec l'objet
            i. Rien
    B. L'inventaire du joueur est plein
        a. Le joueur interagit avec l'objet
            i. Un message indique au joueur que son inventaire est plein
        b. Le joueur n'interagit pas avec l'objet
            i. Rien

### Diagramme(s) de séquence

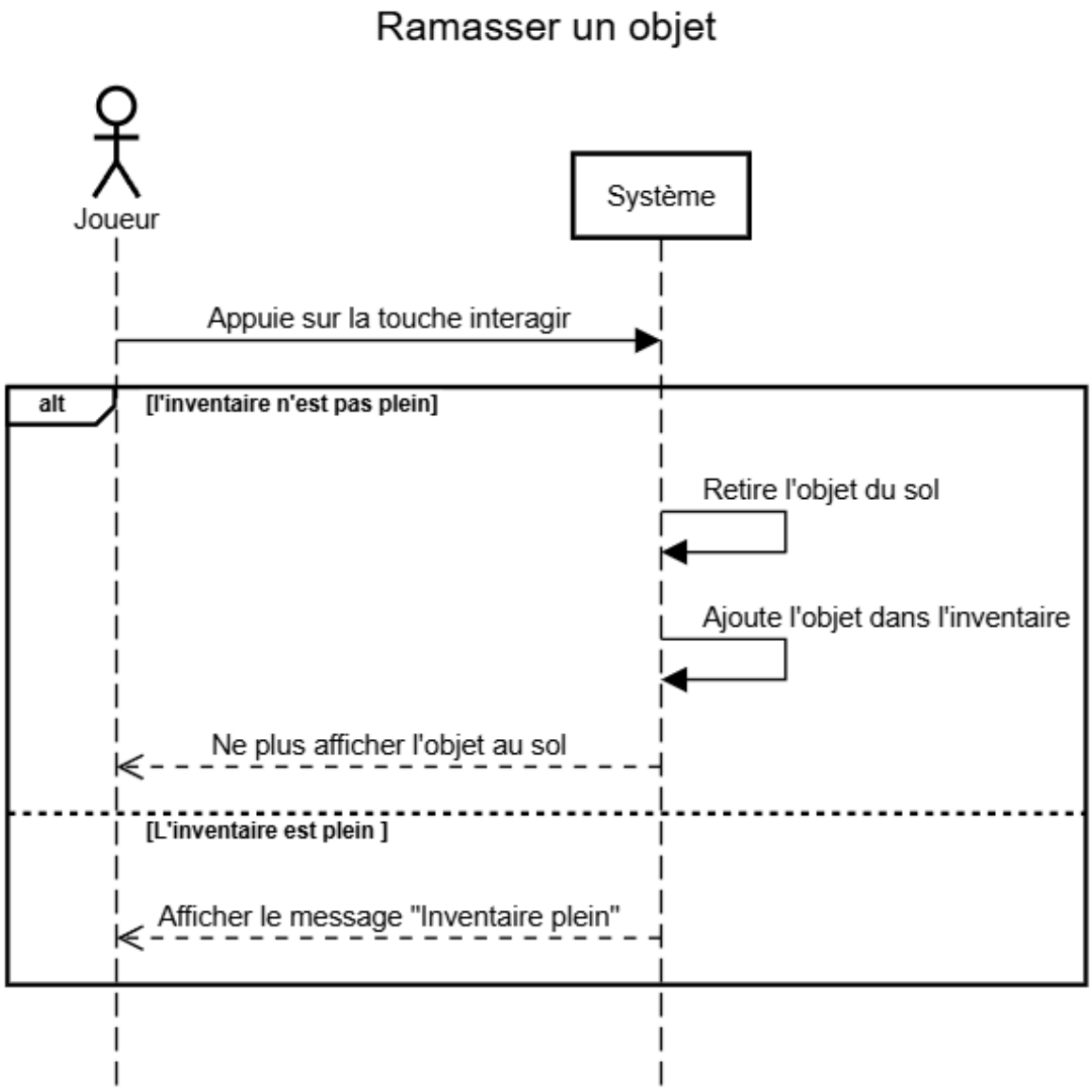



Fig. 19. – Ramasser un objet

# III. Diagramme de classes d'analyse

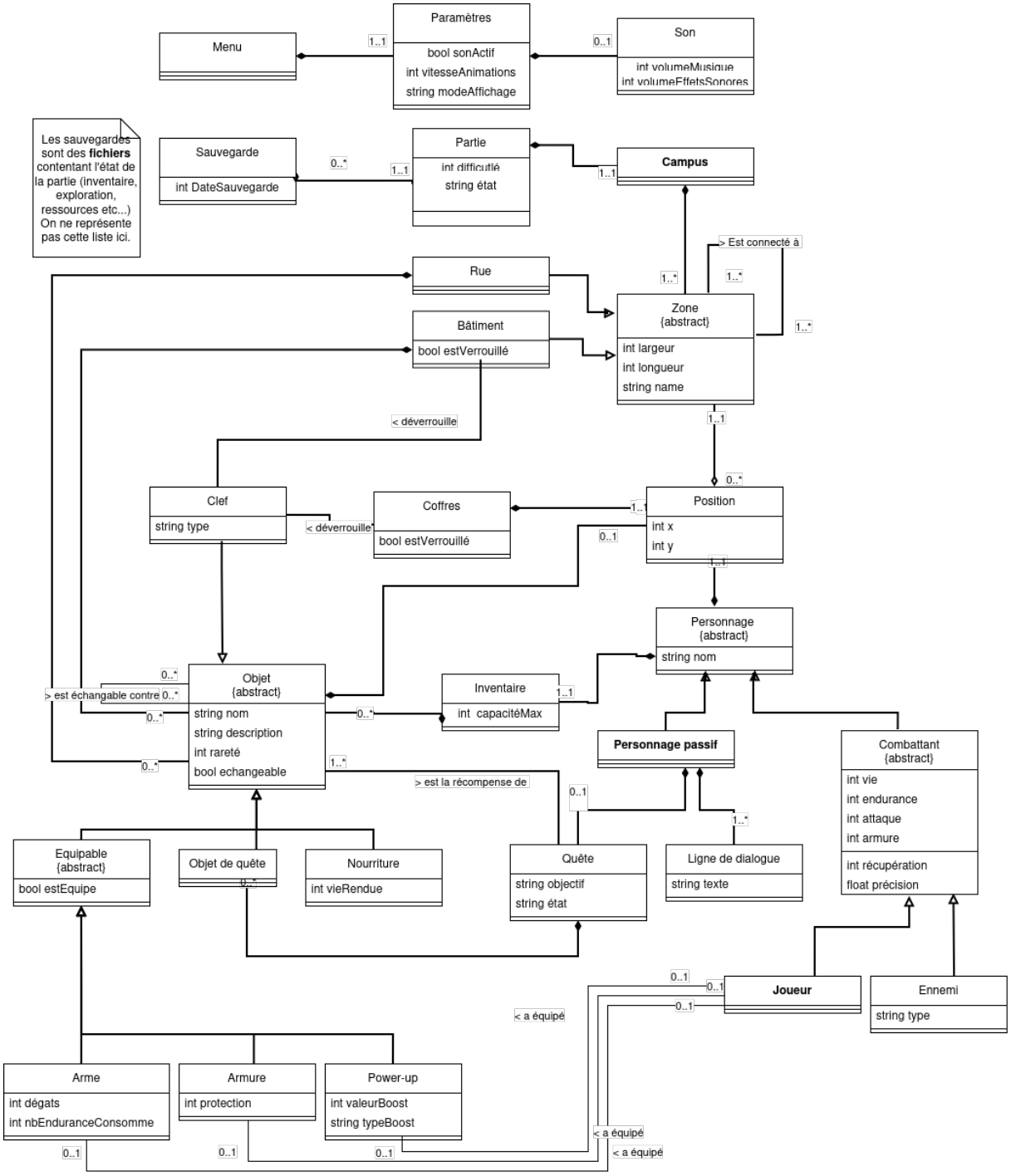



Fig. 20. – Diagramme de classes d'analyse

On représente ci-dessus les différentes classes d'analyse correspondant au cahier des charges précédent. On établit également les liens entre ces différentes classes.

# IV. Architecture logique du logiciel

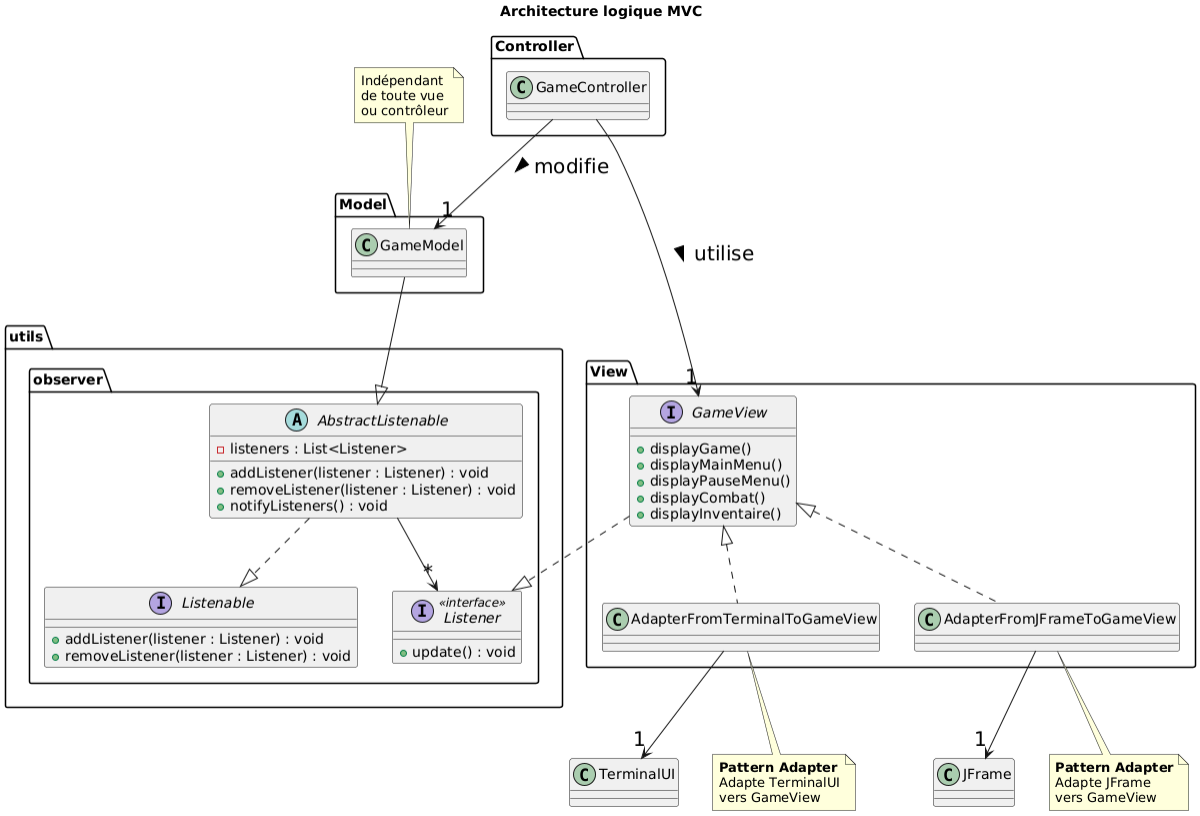



Fig. 21. – Architecture logique

Notre architecture repose sur le modèle **MVC** (Modèle-Vue-Contrôleur), combiné avec les design patterns **Observer** et **Adapter**. Elle est organisée en quatre packages principaux.

### Package Model

Contient la logique métier du jeu. Ce package est totalement indépendant de la vue et du contrôleur : il ne connaît ni la manière dont le jeu est affiché, ni la manière dont les entrées utilisateur sont lues. Lorsque son état change (déplacement du joueur, objet ramassé, changement de zone…), il notifie ses observateurs via le pattern Observer.

### Package Controller

Il sert d'intermédiaire entre la vue et le modèle. Il reçoit les actions de l'utilisateur depuis la vue, les interprète, et modifie le modèle en conséquence. Il ne dépend que de l'interface `GameView`, jamais d'une implémentation concrète, ce qui le rend indépendant de tout outil d'affichage.

### Package View

Ce package définit l'interface `GameView` qui déclare les méthodes d'affichage du jeu : `displayGame()`, `displayMainMenu()`, `displayPauseMenu()`, `displayCombat()` et `displayInventaire()`. L'implémentation actuelle, `AdapterFromTerminalToGameView`, adapte une classe `TerminalUI` (entrée/sortie console) vers cette interface grâce au pattern Adapter.

### Package Utils (Observer)

Contient le mécanisme d'observation réutilisable : l'interface `Listenable`, la classe abstraite `AbstractListenable` (qui gère la liste des listeners et leur notification), et l'interface `Listener` avec

sa méthode `update()`. Le model étend `AbstractListenable` et la vue implémente `Listener`, ce qui permet au modèle de notifier la vue à chaque changement d'état sans la connaître directement.

**Comment ajouter une interface graphique**

L'architecture a été conçue pour rendre ce changement simple. Pour remplacer l'interface textuelle par une interface graphique (Swing, JavaFX…), il suffit de :

1. Créer une classe `AdapterFromJFrameToGameView` qui implémente l'interface `GameView` en déléguant l'affichage à un `JFrame` (ou tout autre composant graphique).
2. Injecter cette nouvelle classe à la place de `AdapterFromTerminalToGameView` au démarrage de l'application.

Aucune modification n'est nécessaire dans les packages Controller, Model. Le pattern Adapter permet d'adapter n'importe quelle outil d'IHM vers le contrat défini par `GameView`, et le pattern Observer garantit que la nouvelle vue sera notifiée des changements du modèle exactement comme l'ancienne. Cette extensibilité est rendue possible par le fait que le contrôleur et le modèle ne dépendent que d'abstractions (`GameView`, `Listener`) et jamais d'implémentations concrètes, conformément au principe d'inversion des dépendances.

# V. Conception de l'incrément

## Incrément choisi

Les fonctionnalités suivantes font partie de l'incrément choisi :

- Lancement d'une partie
- Déplacement entre les différentes zones (rues et bâtiments)
- Ramassage, équipement et jet des objets (donc la gestion de l'inventaire)
- Ouverture des bâtiments

Elles seront dans un premier temps mises à oeuvre à travers un terminal.

Si le temps le permet, seront développés :

- Interface graphique
- Combat contre des personnages hostiles (ennemis) et niveau de difficulté
- Echange avec des personnages passifs
- Coffres
- Fin de jeu
- Paramètres
- Musique
- Pause
- Sauvegarde et chargement
- Aides et règles

## Diagramme de classes logicielles

### Vue d'ensemble MVC

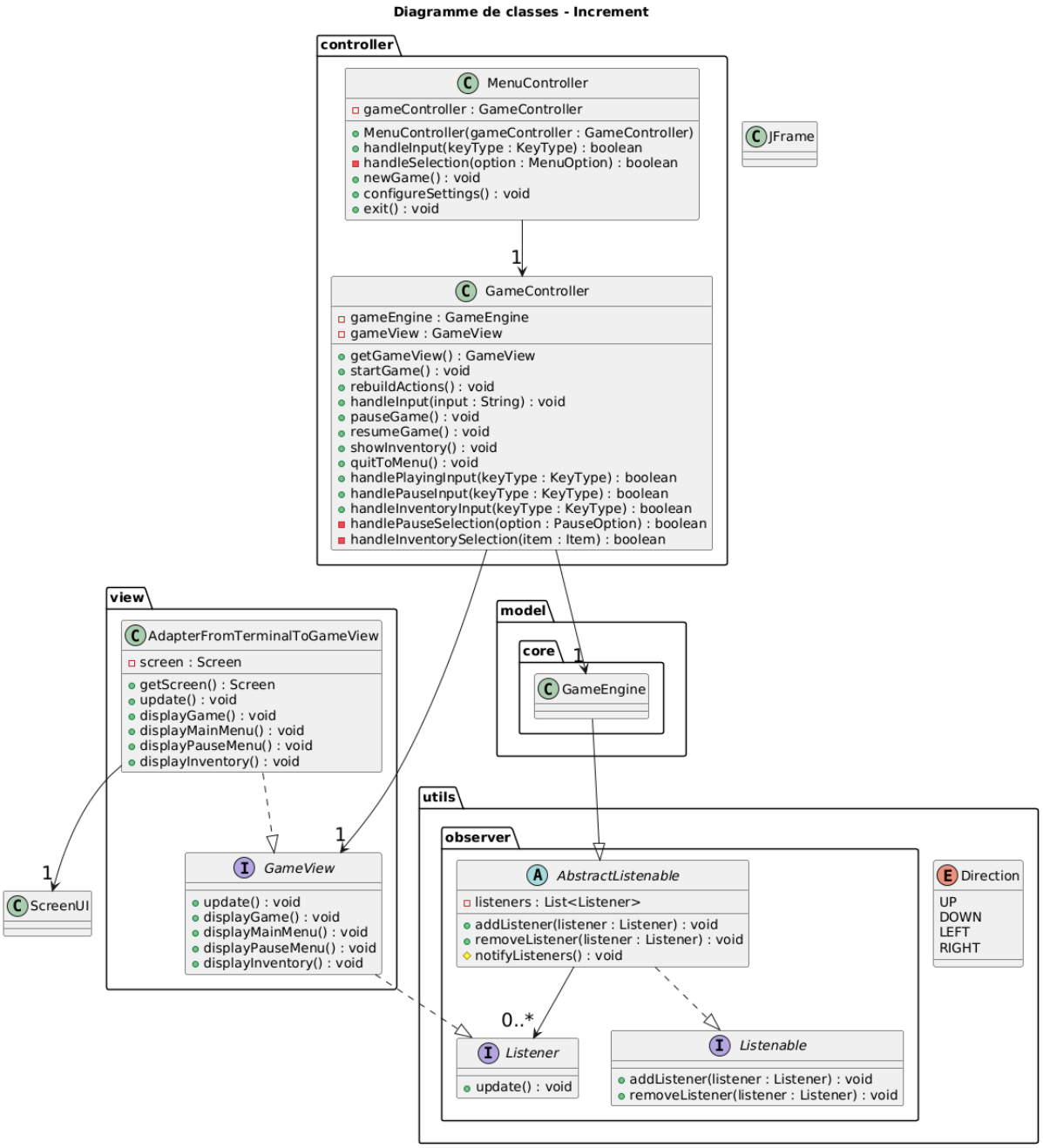



Fig. 22. – Vue d'ensemble MVC

Ce diagramme présente l'assemblage global des packages et les patterns mis en œuvre.

Le *pattern Adapter* est utilisé côté vue : l'interface **GameView** définit le contrat d'affichage, et **AdapterFromTerminalToGameView** adapte la classe **ScreenUI** (console) vers cette interface.

Le *pattern Observer* relie le modèle à la vue sans couplage direct. **GameEngine** étend **AbstractListenable** et peut notifier ses **Listener**. **GameView** implémente **Listener** : quand le modèle change, il appelle **notifyListeners()** et la vue se met à jour via **update()**.

Le *pattern Singleton* est appliqué sur **GameEngine** via **getInstance()**, garantissant un point d'accès unique au moteur de jeu.

Côté contrôleur, **MenuController** gère le menu principal et délègue au **GameController**, qui dispatche les entrées selon l'état courant.

### Package model.core

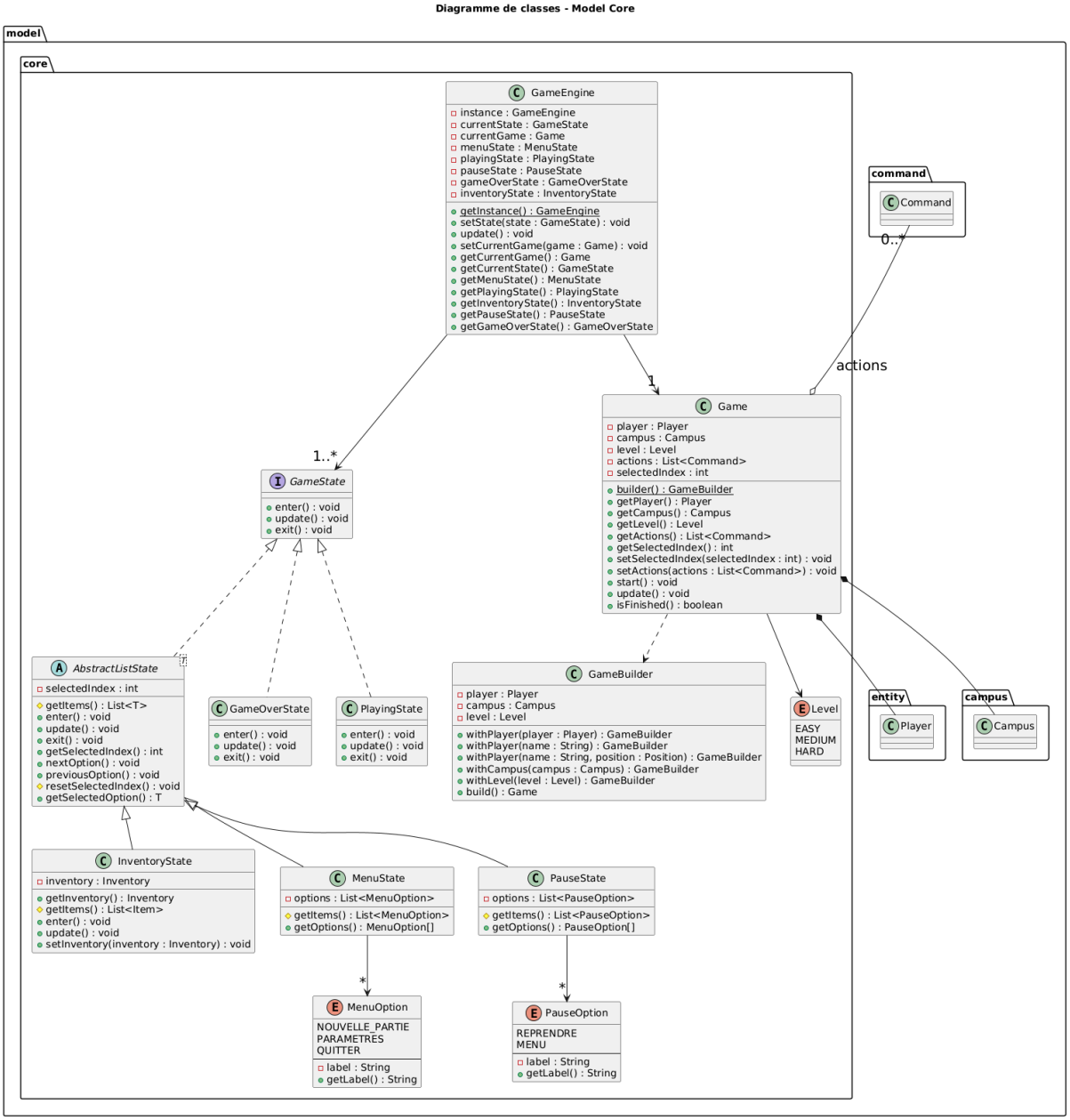



Fig. 23. – Package core

Ce package constitue le cœur du modèle et concentre trois design patterns.

Le *pattern State* gère les différents états du jeu. L'interface **GameState** définit le contrat que chaque état doit respecter. Cinq états concrets l'implémentent : **MenuState**, **PlayingState**, **PauseState**, **InventoryState** et **GameOverState**. **GameEngine** détient l'état courant et lui délègue le comportement, éliminant tout switch/case.

La classe abstraite **AbstractListState** factorise la logique commune des états qui présentent une liste navigable (menu, pause, inventaire). **MenuState**, **PauseState** et **InventoryState** en héritent.

Le *pattern Builder* est utilisé pour la création d'une partie. **GameBuilder** permet de construire un objet **Game** de manière fluide, évitant un constructeur avec de nombreux paramètres.

La classe **Game** détient une liste de **Command**, faisant le lien avec le *pattern Command* défini dans le package suivant.

Les enums **MenuOption** et **PauseOption** définissent les options disponibles dans chaque menu, chacune portant un label affichable.

### Package model.command

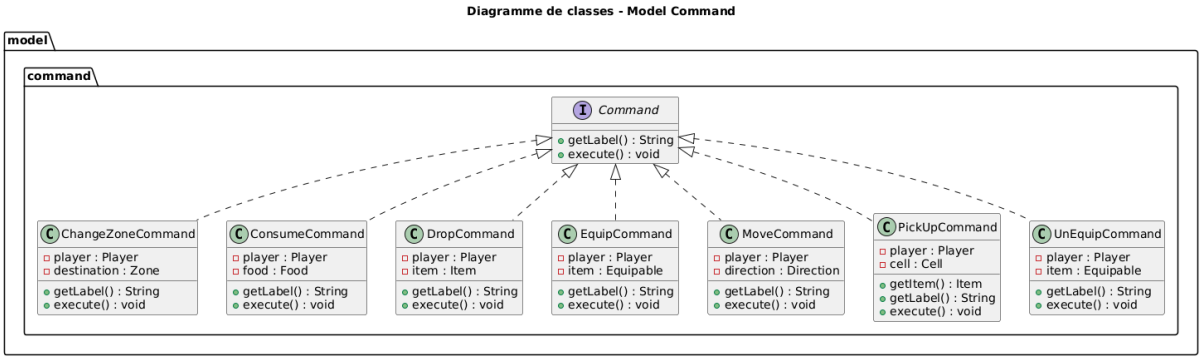



Fig. 24. – Package command

Ce package implémente le *pattern Command*. L'interface **Command** définit deux méthodes : **execute()** pour exécuter l'action et **getLabel()** pour obtenir un libellé affichable dans la vue.

Sept commandes concrètes encapsulent chacune une action du jeu : **MoveCommand** (déplacement), **PickUpCommand** (ramassage d'objet), **EquipCommand** et **UnEquipCommand** (équipement), **ConsumeCommand** (consommation de nourriture), **DropCommand** (jeter un objet) et **ChangeZoneCommand** (changement de zone).

Chaque commande détient une référence vers le **Player** et délègue la logique métier au modèle: la commande orchestre, le modèle exécute. Ce pattern permet d'ajouter de nouvelles actions sans modifier le code existant et évite toute logique conditionnelle complexe dans le contrôleur.

### Package model.world

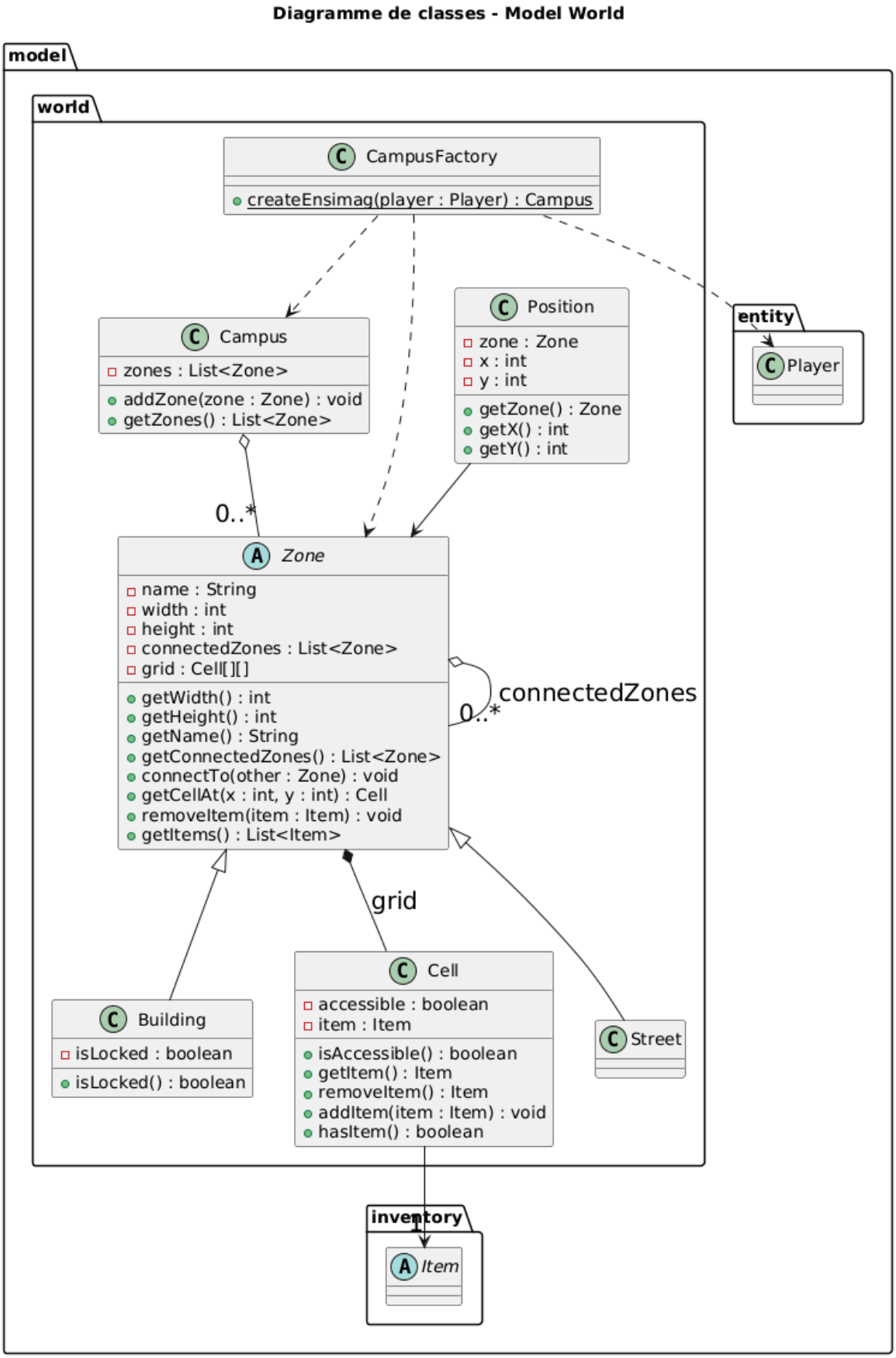



Fig. 25. – Package world

Ce package modélise l'environnement du jeu.

Le *pattern Factory* est utilisé via **CampusFactory**, qui encapsule la logique de création du campus ENSIMAG. Sa méthode **createEnsimag()** construit et retourne un **Campus** complet avec ses zones, cellules et objets positionnés. Cela isole la logique d'initialisation du monde et permet de créer facilement d'autres cartes à l'avenir.

**Campus** agrège une liste de **Zone**. **Zone** est une classe abstraite avec deux sous-classes : **Building** (qui possède un attribut **isLocked**) et **Street**. Chaque zone contient une grille de **Cell**, où chaque cellule sait si elle est accessible et si elle contient un **Item**. L'auto-association **connectedZones** modélise les connexions entre zones, permettant la navigation dans le campus.

**Position** localise le joueur avec ses coordonnées (x, y) et une référence vers la **Zone** courante, ce qui permet aux commandes d'accéder directement à la grille sans dépendance supplémentaire.

### Package model.inventory

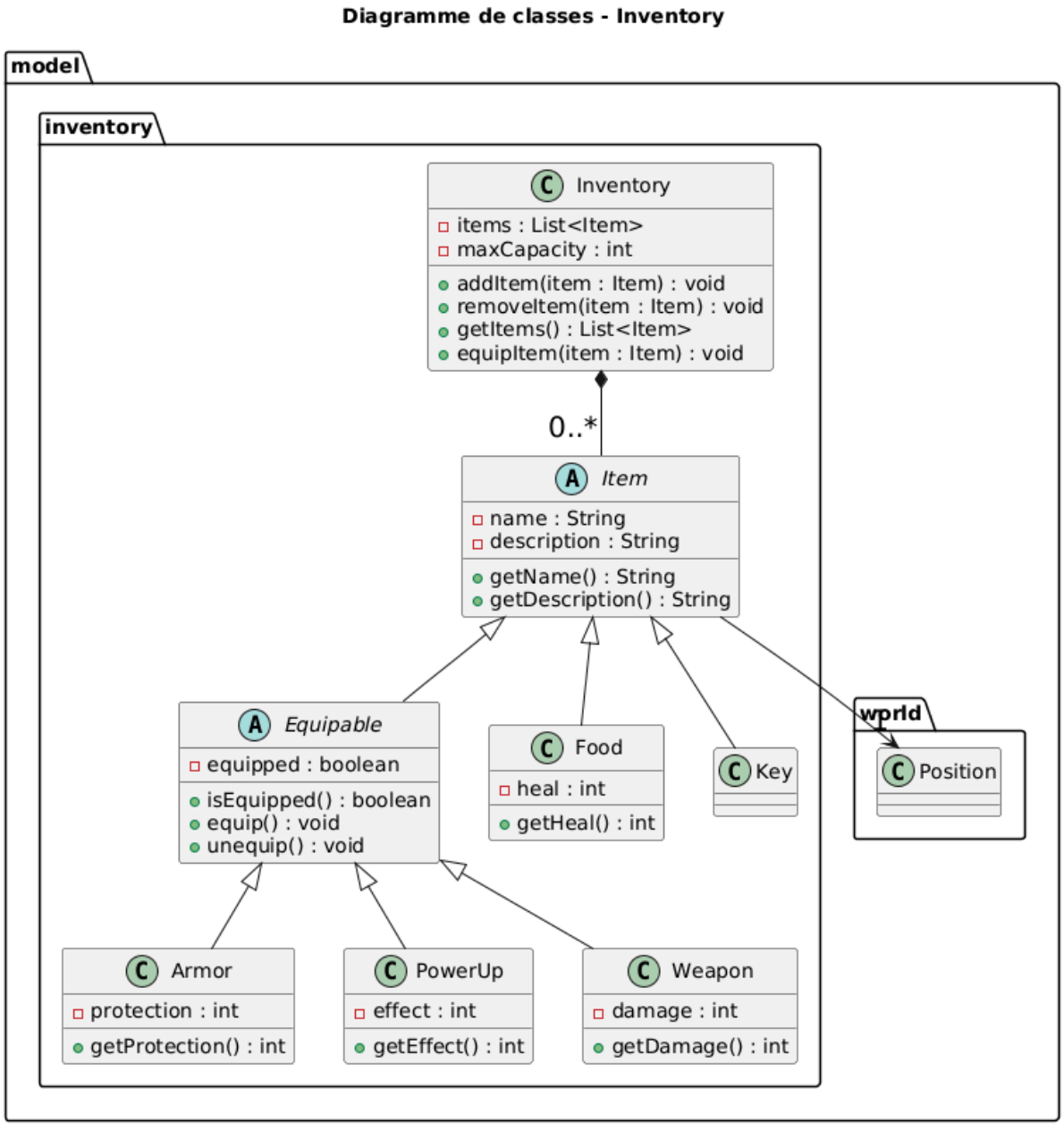



Fig. 26. – Package inventory

Ce package modélise le système d'inventaire et la hiérarchie des objets.

**Inventory** gère une liste d'**Item** avec une capacité maximale. Il expose les opérations d'ajout, de suppression et d'équipement d'objets.

La hiérarchie des objets repose sur la classe abstraite **Item**, spécialisée en deux branches. La branche **Equipable** (classe abstraite) regroupe les objets que le joueur peut porter : **Weapon** (dégâts), **Armor** (protection) et **PowerUp** (effet de boost). Chaque équipable gère son propre état via les méthodes **equip()** et **unequip()**. La seconde branche contient **Food** (restauration de vie) et **Key** (déverrouillage de bâtiments), qui sont des objets consommables ou utilitaires non équipables.

Cette hiérarchie permet au pattern Command de manipuler les objets de manière polymorphe : **EquipCommand** travaille avec n'importe quel **Equipable**, **ConsumeCommand** avec un **Food**, sans connaître le type concret.

**Package model.entity**

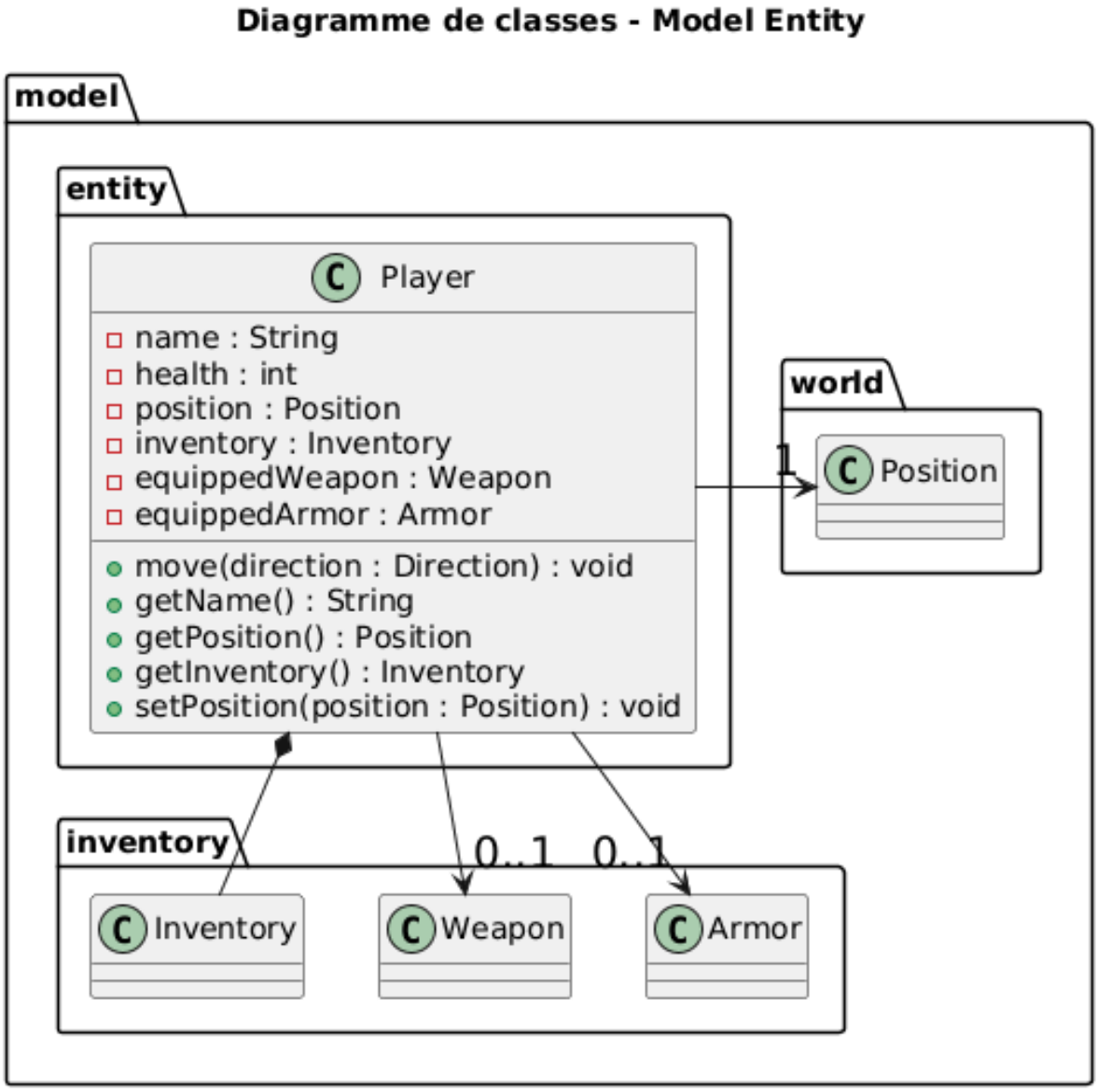



Fig. 27. – Package entity

Ce package contient la classe **Player**, qui représente le joueur. Elle détient ses attributs (nom, vie), sa **Position**, son **Inventory** et ses emplacements d'équipement : **equippedWeapon** et **equippedArmor**.

La méthode **move()** prend une **Direction** en paramètre et modifie la position du joueur. C'est cette méthode que **MoveCommand** appelle; la commande orchestre (vérification des conditions), le modèle exécute (mise à jour de la position).

**Diagramme d'états-transitions**

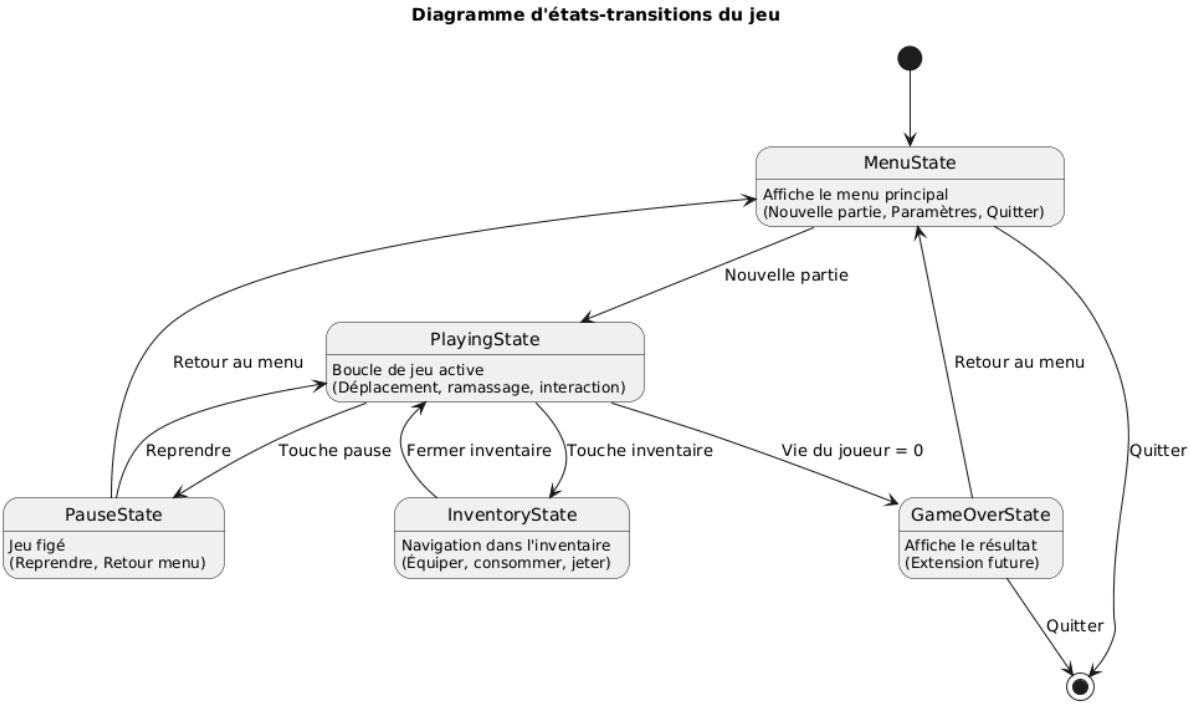



Fig. 28. – Diagramme d'états-transions

Ce diagramme illustre les transitions entre les différents états du jeu, implémentés via le *pattern State*.

Au lancement, le jeu entre dans l'état **MenuState**. Depuis le menu, le joueur peut démarrer une nouvelle partie (transition vers **PlayingState**) ou quitter l'application.

**PlayingState** est l'état principal : le joueur se déplace, ramasse des objets et interagit avec l'environnement. Depuis cet état, il peut ouvrir l'inventaire (transition vers **InventoryState**) ou mettre le jeu en pause (transition vers **PauseState**).

**InventoryState** permet de naviguer dans l'inventaire (équiper, consommer, jeter). La fermeture de l'inventaire ramène à **PlayingState**.

**PauseState** fige le jeu et offre deux options : reprendre la partie (retour à **PlayingState**) ou revenir au menu principal (retour à **MenuState**).

**GameOverState** est prévu pour une extension future (combat). Il sera atteint lorsque la vie du joueur tombe à zéro, et permettra de revenir au menu ou de quitter.

Chaque transition correspond à un appel à **setState()** dans le **GameEngine**, qui exécute **exit()** sur l'état courant, puis **enter()** sur le nouvel état.

## Diagrammes de séquence

Les différentes parties de notre programme sont très interconnectées entre elles, c'est notamment particulièrement visible sur les exemples d'actions suivants que nous avons illustrés par des diagrammes de séquence.

Ces diagrammes représentent les interactions entre les différents objets du système lors de l'exécution de certaines actions.

Lorsque des flèches sont encadrées par une boite portant le nom d'une fonction (comme `notifyListeners()` par exemple) cela signifie que ces interactions sont effectuées au sein de cette fonction (sur plusieurs niveaux de profondeur).

Lorsqu'elles sont encadrées par une boite de la forme

```
alt <cond1>

----
<cond2>
```

les 1ères interactions ne sont effectuées que lorsque `<cond1>` est vraie, les 2èmes que si `<cond2>` est vraie.

Les objets sont regroupés en 3 groupes : **Controller**, **Model** ou **Vue** en fonction de quelle partie de l'architecture ils appartiennent, ce qui permet d'illustrer les interactions entre ces partie.

### Initialisation d'une partie

Ce diagramme représente les interactions entres les objets du système lorsque l'utilisateur initialise une partie depuis le menu principal.

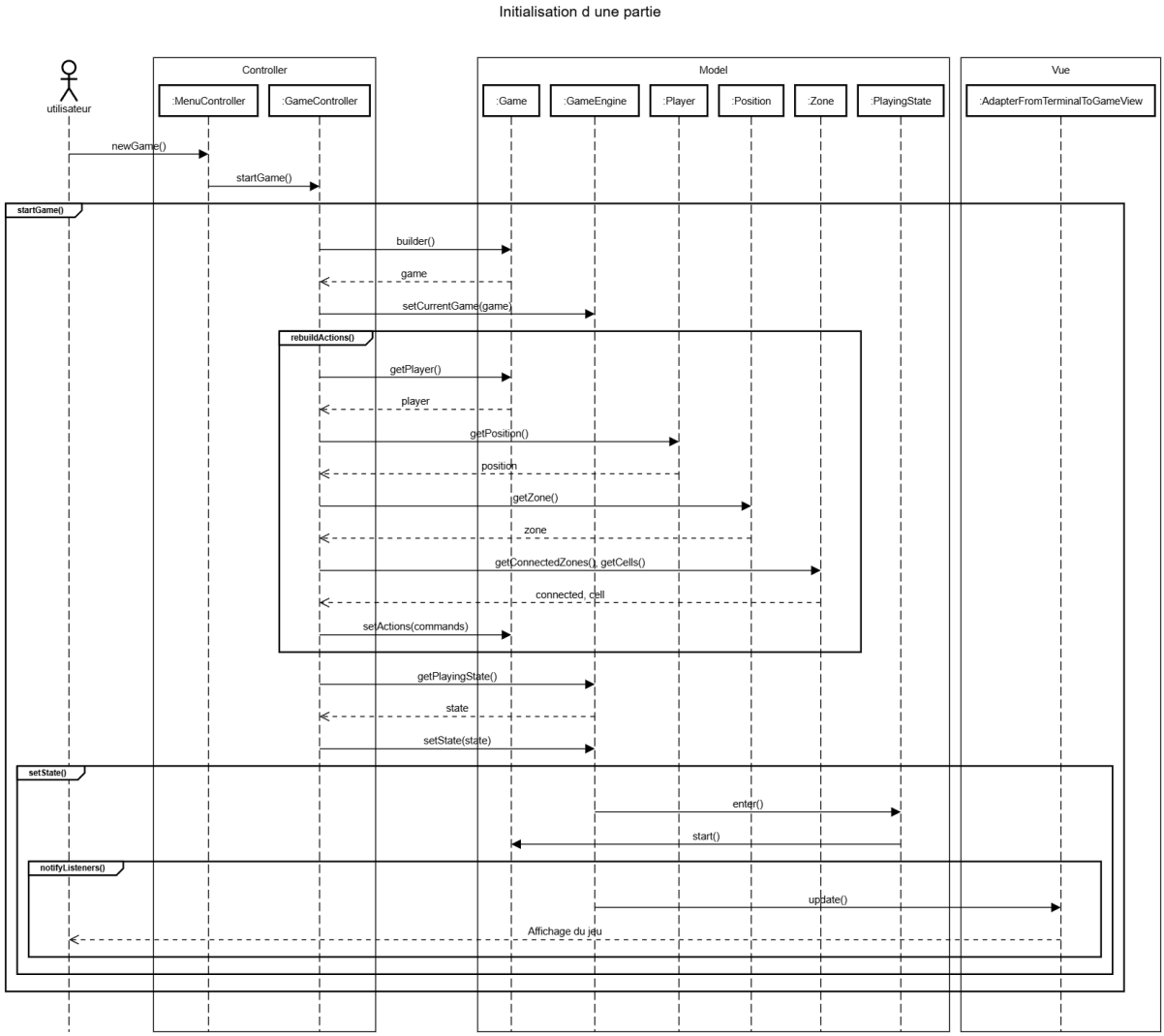



Fig. 29. – Initialisation d'une partie

### Consommation de nourriture

Ce diagramme représente les interactions entres les objets du système lorsque l'utilisateur consomme de la nourriture depuis son inventaire.

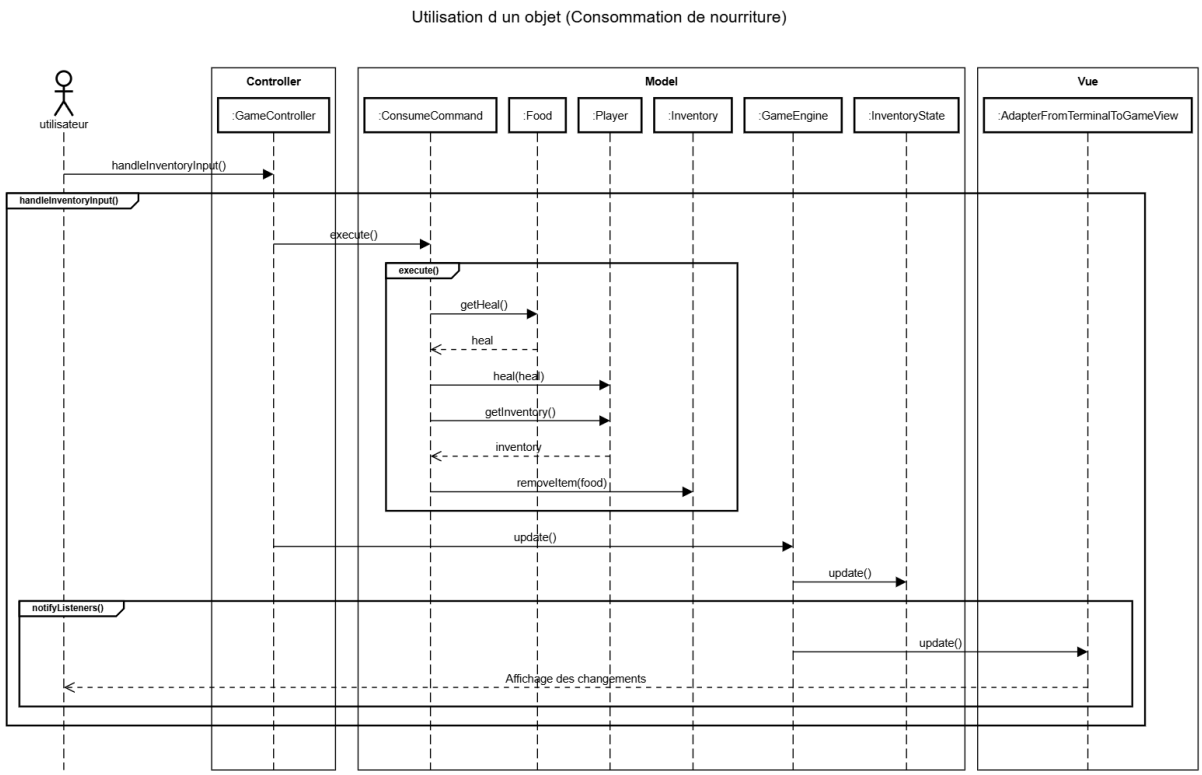



Fig. 30. – Utilisation d'un objet (Consommation de nourriture)

**Ramasser un objet**

Ce diagramme représente les interactions entres les objets du système lorsque l'utilisateur ramasse un objet au sol en jeu.

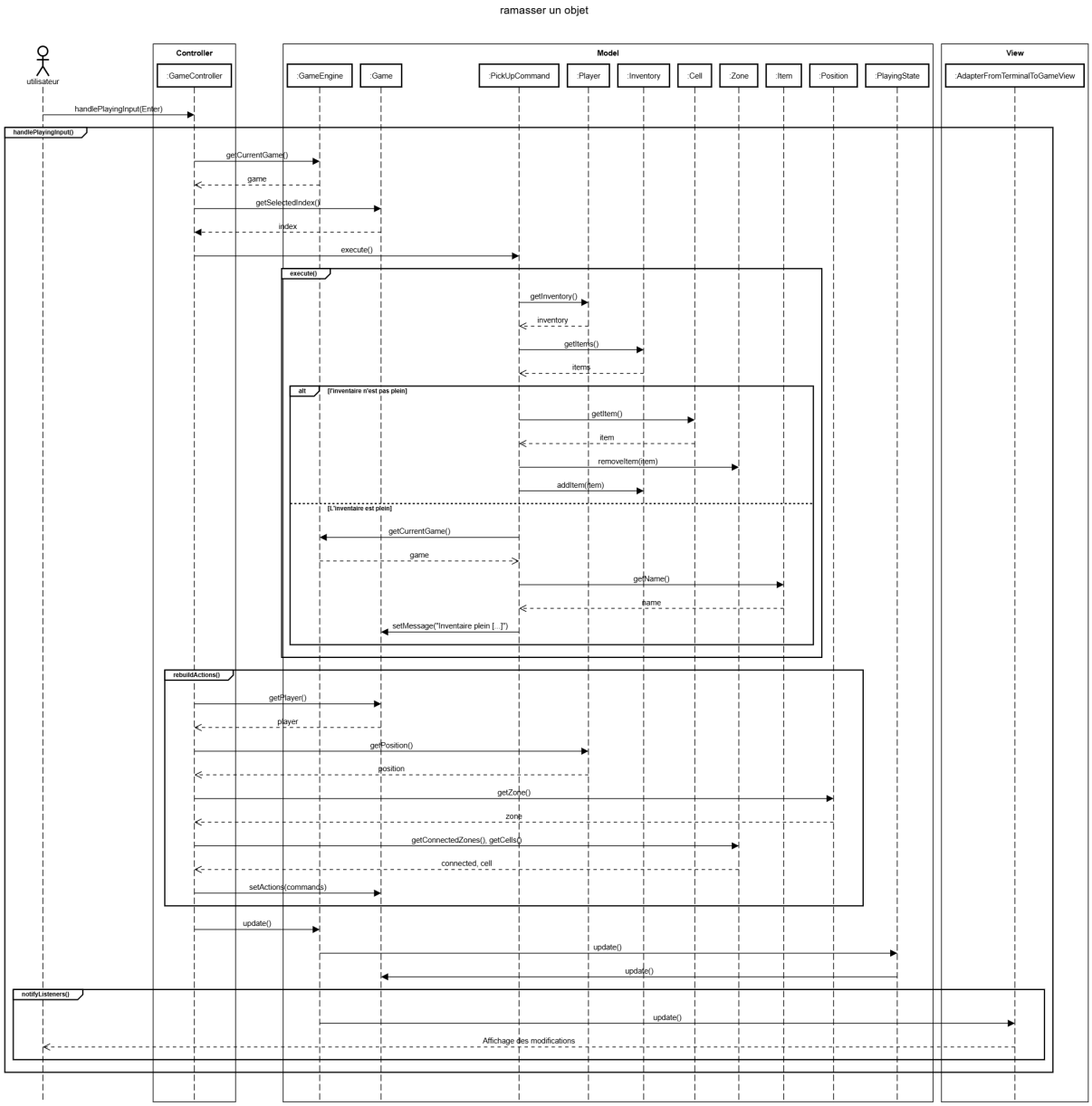



Fig. 31. – Ramasser un objet

### Déverouiller un bâtiment

Ce diagramme représente les interactions entres les objets du système lorsque l'utilisateur déverrouille en jeu. On suppose ici que l'utilisateur possède la clef du bâtiment dans son inventaire.

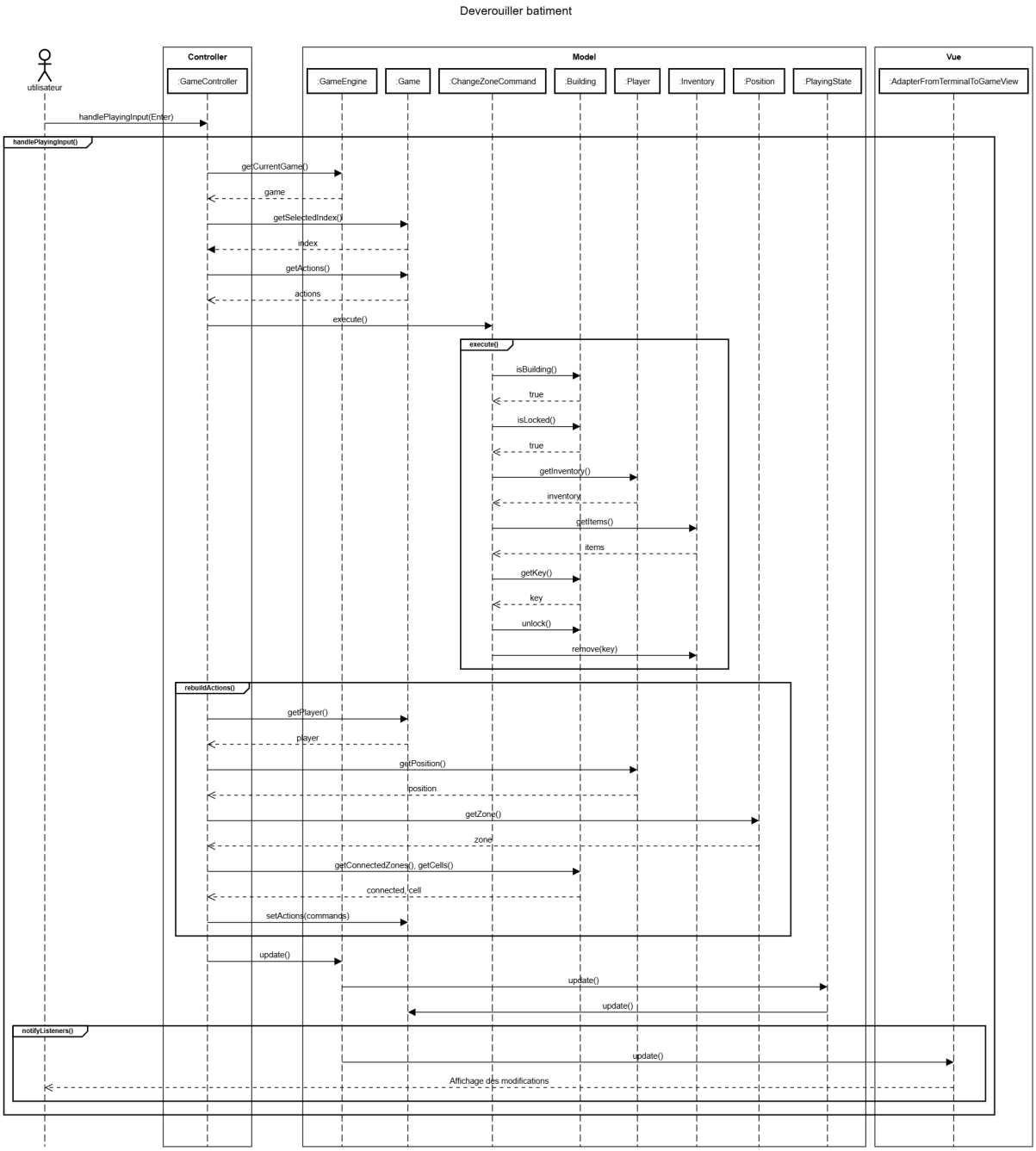



Fig. 32. – Déverrouiller bâtiment

**Déplacement entre zones.png**

Ce diagramme représente les interactions entres les objets du système lorsque l'utilisateur se déplace d'une zone à une autre en jeu.

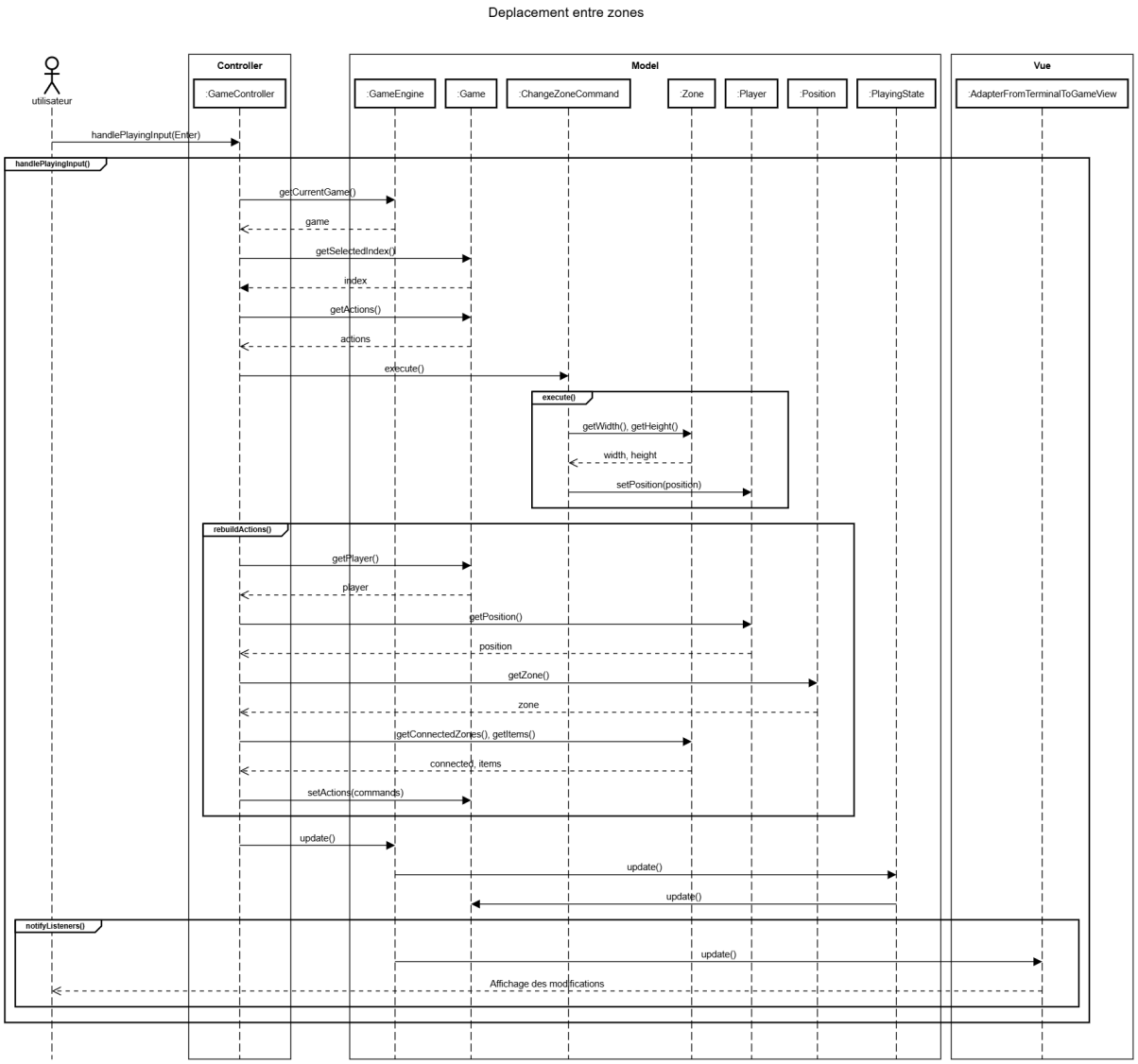



Fig. 33. – Déplacement entre zones

# VI. Manuel utilisateur

## Lancement

Pour compiler et lancer le jeu :

```
make compile
make run
```

Le jeu s'ouvre dans le terminal courant.

## Navigation générale

Le jeu se joue entièrement au clavier. Toutes les interfaces partagent les mêmes principes :

| Touche | Action |
|---|---|
| Flèche du Haut / Flèche du Bas | Naviguer dans une liste |
| Entrée | Valider la sélection |
| Échap | Retour / Pause |

### Menu principal

Au lancement, le joueur arrive sur le menu principal qui propose trois options :

Nouvelle Partie : lance une nouvelle partie Paramètres : accès aux réglages Quitter : ferme le jeu

### Écran de jeu

L'écran affiche le nom de la zone actuelle ainsi que la liste des actions disponibles. Deux types d'actions sont possibles :

Aller vers [zone] : se déplacer vers une zone connectée. Si le bâtiment est verrouillé, il faut posséder la clé correspondante dans l'inventaire. Ramasser [objet] : récupérer un objet présent dans la zone et le placer dans l'inventaire.

| Touche | Action |
|---|---|
| Flèche du Haut / Flèche du Bas | Naviguer dans la liste des actions |
| Entrée | Exécuter l'action sélectionnée |
| Échap | Ouvrir le menu pause |
| F1 | Ouvrir l'inventaire |

### Inventaire

L'inventaire (accessible via F1) affiche les objets du sac. Sélectionner un objet permet de voir son type, sa description et les actions possibles.

| Touche | Action |
|---|---|
| Entrée | Utiliser l'objet (équiper ou consommer) |
| F2 | Jeter l'objet |
| Échap | Retour au jeu |

Le comportement varie selon le type d'objet :

Arme, Armure, Power-Up : s'équipe dans le slot correspondant. Si un objet du même type est déjà équipé, il retourne dans l'inventaire. Nourriture : consommée immédiatement, restaure des points de vie. Clé : utilisée automatiquement lors de l'accès à un bâtiment verrouillé.

### Pause

Appuyer sur Échap pendant le jeu ouvre le menu pause, qui propose deux options :

- Reprendre : retour au jeu
- Retour au menu principal : quitte la partie en cours

## VII. Bilan sur les outils de modélisation

### PlantUML

PlantUML a été utilisé pour la réalisation des diagrammes de classes et du diagramme d'états-transitions.

**Problèmes rencontrés et solutions :**

- Les multiplicités étaient peu lisibles sur les diagrammes complexes. Solution : les écrire explicitement entre guillemets sur chaque relation et augmenter la taille de police via *skinparam defaultFontSize*.

- Le moteur de layout produit parfois des agencements peu lisibles avec de nombreuses classes. Solution : découper le diagramme de classes détaillé en sous-diagrammes par package (core, command, world, inventory, entity).

**Bilan :** PlantUML est adapté pour des diagrammes de taille modérée. L'approche textuelle facilite les modifications rapides. Sa principale limite reste le contrôle du layout, contournée par le découpage en sous-diagrammes.